

OS 1

🕒 Created @July 8, 2025 10:46 PM

Session 1: Introduction to Operating System

1 What is an Operating System (OS)?

Definition:

An **Operating System** is *system software* that acts as an **intermediary** between the **user/application programs** and the **computer hardware**.

Core Purpose:

- It **manages hardware resources** (CPU, memory, disks, I/O devices).
- It provides an **environment** where users can run programs **easily and efficiently**.

Examples of OS:

- Windows
- Linux
- macOS
- Android
- iOS

How is OS different from Application Software?

Operating System	Application Software
System software controlling the overall working of the computer	Software that helps perform specific tasks
Runs from the time computer is switched on till shut down	Runs only when user starts it
Examples: Windows, Linux	Examples: MS Word, Photoshop
Manages hardware resources	Uses OS services to access hardware
Hardware-dependent	Mostly hardware-independent (if OS supports it)

✓ Key Difference:

- **OS provides services and environment for application software to run.**
- Application software **cannot** run **without** the OS.

! Why is the OS hardware dependent?

Because:

1. **Device Drivers:** OS must interact directly with **hardware controllers** (disk controllers, display controllers, etc.).
 - For example, the OS must know how to control the hard disk to read/write data.
2. **Instruction Set Architecture (ISA):** OS kernel code depends on processor instructions (Intel x86, ARM, etc.).
 - An OS compiled for Intel will **not** work on ARM.
3. **Memory Management:** OS must know how the hardware organizes RAM.
4. **Interrupt Handling:** OS must know how hardware generates interrupts.

Bottom line:

👉 The OS has to understand the **low-level working of hardware**, so it is closely tied to the hardware platform.

2 Different Components of OS

An OS is a **collection of components** working together. Here are the main ones:

1. Kernel

- **Core part of OS.**
- Always loaded in memory (resident).
- Manages:
 - CPU scheduling
 - Memory allocation
 - Process management
 - Device management

Examples:

- monolithic kernel (Linux)
 - microkernel (Minix)
-

2. Process Management

- Responsible for:
 - Creating and terminating processes
 - Scheduling processes (which process runs when)
 - Managing process states (Ready, Running, Blocked)
 - Ensures fair and efficient CPU utilization.
-

3. Memory Management

- Allocates and deallocates memory space.
 - Keeps track of **which part of memory is used by which process.**
 - Handles swapping between RAM and disk (Virtual Memory).
-

4. File System Management

- Organizes and manages files and directories.
 - Provides:
 - File creation
 - Deletion
 - Read/Write access
 - Security and permissions
-

5. Device Management (I/O System)

- Manages I/O devices using **device drivers**.
 - Buffers, caches, queues I/O requests.
-

6. Security and Protection System

- Controls **access to resources**.
 - User authentication (login passwords).
 - Permissions for files and devices.
-

7. User Interface

- Command Line Interface (CLI)
 - e.g., Linux shell
 - Graphical User Interface (GUI)
 - e.g., Windows desktop
-

Mnemonic to remember major components:

Kernel

Process Management

Memory Management

File System

Device Management

Security

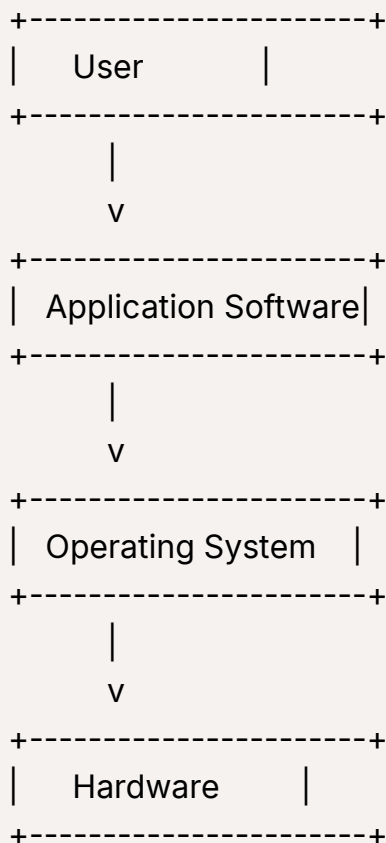
User Interface

👉 KPM-FDSU

3 Basic Computer Organization Required for OS

Before understanding OS, you must know **computer organization**, because OS interacts with hardware.

Here's a **conceptual layout**:



🟢 Basic Hardware Components:

✅ CPU (Central Processing Unit):

- Executes instructions.
- Generates interrupts.

- Needs scheduling.

✓ **Main Memory (RAM):**

- Stores data and instructions for currently running programs.
- OS allocates and deallocates RAM.

✓ **Secondary Storage:**

- Hard disk, SSD.
- OS provides file systems to manage storage.

✓ **I/O Devices:**

- Keyboard, mouse, printer, network cards.
- Managed by OS using device drivers.

✓ **Bus:**

- Communication path between CPU, memory, devices.

✓ **Registers:**

- Small memory inside CPU (Program Counter, Stack Pointer, etc.)
- OS uses them during context switching.

🟢 **OS Role in Basic Organization:**

1. Resource Allocation:

- Decides which process gets CPU, memory, I/O.

2. Process Control:

- Switches between processes (multitasking).

3. Memory Management:

- Maps logical addresses to physical addresses.

4. Device Control:

- Accepts I/O requests and interacts with hardware.

✓ **Example of Interaction:**

1. A user clicks a program icon.
2. OS loads program from disk into memory.
3. CPU executes program instructions.
4. Program requests to read a file.
5. OS uses device drivers to get data from disk.

✨ Quick Recap Table:

Topic	Key Points
What is OS	Intermediary between user/applications and hardware
OS vs Application Software	OS manages hardware; apps perform tasks
Why hardware dependent	Needs drivers, instruction set, interrupt handling
OS Components	Kernel, Process Mgmt, Memory Mgmt, File System, Device Mgmt, Security, UI
Basic Computer Organization	CPU, Memory, Storage, I/O devices, Bus, Registers

✅ Tip for MCQs:

Watch for questions like:

- "Which of the following is not an OS component?"
- "Which component manages memory allocation?"
- "Why is OS hardware dependent?"
- "What is the difference between OS and application software?"

🟢 Comparison: How Are These Different and Why?

Below is a structured comparison table to help you **differentiate** easily (great for MCQs):

Type of OS	Purpose	Key Features	Examples
Desktop OS	General-purpose computing	Rich GUI, multimedia, multitasking	Windows 11, macOS, Ubuntu
Mobile OS	Smartphones, tablets	Touch interface, battery optimization	Android, iOS
Embedded OS	Dedicated devices with limited tasks	Small footprint, hardware-specific	VxWorks, Embedded Linux
Real-Time OS (RTOS)	Time-critical systems	Deterministic response, real-time tasks	QNX, FreeRTOS, RTEMS
Server OS	Serving clients over network	High stability, security, scalability	Windows Server, Red Hat Linux

✓ Reasons for Differences:

1 Target Hardware:

- Desktop: Powerful CPUs, GPUs, lots of RAM.
- Mobile: Limited battery, ARM CPUs.
- Embedded: Minimal resources (few KBs of memory).
- Servers: Multi-core CPUs, huge RAM, redundant storage.

2 User Interaction:

- Desktop: Keyboard + mouse, rich GUI.
- Mobile: Touchscreen.
- Embedded: No user interface or tiny screens.
- Server: Often no GUI, command line admin.

3 Real-Time Requirements:

- RTOS: Must guarantee strict timing.
- Desktop/Mobile: Best-effort response acceptable.

- Embedded: May or may not be real-time.

4 Power Management:

- Mobile OS: Aggressive battery saving.
- Desktop/Server: Less constrained.
- Embedded: Sometimes battery-powered, highly optimized.

5 Security and Reliability:

- Servers: Highest security & uptime requirements.
- Mobile: Strong app sandboxing.
- Embedded: Often isolated systems, security can be lighter.

Quick Recap Table (MCQ Cheat Sheet)

OS Type	Example	Distinct Feature
Desktop	Windows 10, Ubuntu	Rich GUI, general computing
Mobile	Android, iOS	Touch input, battery optimization
Embedded	FreeRTOS, VxWorks	Minimal footprint, specific purpose
RTOS	QNX, RTEMS	Deterministic timing, hard real-time constraints
Server	Red Hat Linux, Windows Server	Multi-user, scalable, secure network services

MCQ Triggers:

- *"Which OS is real-time?"* → QNX, VxWorks
- *"Which OS is used in smartphones?"* → Android, iOS
- *"Which is NOT a server OS?"* → Windows 10 (desktop)
- *"What OS powers embedded devices?"* → FreeRTOS, Embedded Linux
- *"Why do embedded OS differ?"* → Small footprint, hardware-specific

Function of OS

Function	Description
Process Management	Creating, scheduling, and terminating processes
Memory Management	Allocating and tracking RAM
File Management	Creating and managing files/directories
Device Management	Controlling I/O devices with drivers
Security & Protection	Authentication, authorization, data safety
User Interface	CLI & GUI interaction with users
Networking	Handling network connections and communication

✨ OS Responsibilities in Device Management

✓ Device Drivers

- Small programs that translate generic OS commands to device-specific actions.

✓ Buffering

- Temporarily storing data during transfer.

✓ Caching

- Keeping frequently used data in memory for fast access.

✓ Spooling

- Storing output (e.g., print jobs) in a queue for sequential processing.

✓ Interrupt Handling

- Devices signal the CPU with interrupts when they finish operations.
- OS handles these interrupts.

✓ I/O Scheduling

- Decides order of servicing I/O requests.

✓ Command Line Interface (CLI)

- Users type commands (e.g., Linux terminal).

✅ Graphical User Interface (GUI)

- Windows, icons, menus, mouse pointers.
-

User Space vs Kernel Space & Modes, Interrupts, and System Calls

1 User Space vs Kernel Space

What is *Space*?

When a process runs, it doesn't directly access the entire physical memory. Instead, the OS separates memory into **two regions**:

✅ User Space

- Where **user applications** run.
- E.g., your programs: Chrome, Word, Python script.
- Restricted access—**cannot directly access hardware**.
- If something goes wrong (like illegal memory access), the OS can terminate it safely.

✅ Kernel Space

- Where the **OS kernel code and data** live.
 - Full control over:
 - CPU instructions
 - Memory
 - Devices
 - Runs in **privileged mode** (more powerful).
-

Why separate them?

- **Protection:** User programs cannot corrupt the OS.
 - **Security:** Prevent malicious apps from controlling hardware.
 - **Stability:** Kernel faults can crash the system, so user code must be isolated.
-

Example:

When you run a C program, its code and data are loaded in **user space**.

But when it calls `printf()`, that eventually uses a **system call** to write to the console, which executes in **kernel space**.

2 User Mode vs Kernel Mode

Closely related to user/kernel space is **mode**:

User Mode

- The CPU executes code with **limited privileges**.
- Can't execute instructions that directly control hardware.
- If it tries, CPU generates an exception (trap).

Kernel Mode (Supervisor Mode)

- Full privileges.
 - Can execute any CPU instruction.
 - Can access any memory area.
 - The OS runs here.
-

How does switching happen?

When your program needs a kernel service, e.g. reading a file:

1. It executes a **system call instruction**.
2. CPU switches from **user mode to kernel mode**.
3. The OS performs the operation.
4. CPU switches back to **user mode** when done.

✓ MCQ Tip:

- “Which mode has full access to hardware?” → Kernel Mode
 - “What happens when user mode code tries to execute privileged instructions?” → Trap/Exception
-

3 Interrupts

What is an Interrupt?

An **interrupt** is a **signal to the CPU** indicating that **something needs attention**.

✓ Sources of interrupts:

- **Hardware Interrupts**
 - From devices (keyboard press, disk I/O completion).
 - **Software Interrupts**
 - From executing programs (e.g., divide by zero error).
 - System calls (sometimes called traps).
-

✨ Types of Interrupts:

✓ 1. Maskable Interrupts

- Can be disabled or ignored by the CPU.

✓ 2. Non-maskable Interrupts

- Cannot be ignored (e.g., hardware failure).

✓ 3. Traps

- Software-generated interrupts (e.g., system calls, exceptions).
-

✨ How do interrupts work?

When an interrupt occurs:

1. CPU stops executing the current instruction.

2. Saves its state (program counter, registers).
 3. Jumps to **Interrupt Service Routine (ISR)**.
 4. ISR handles the event.
 5. CPU resumes previous execution.
-

✓ Example:

When you press a key:

- Keyboard controller sends an interrupt.
 - CPU pauses running code.
 - OS reads the keystroke.
 - CPU resumes.
-

✓ MCQ Tip:

- *"Interrupt from hardware device is called?"* → Hardware interrupt
 - *"Interrupt generated by an error in program?"* → Trap/Exception
-

System Calls

What is a System Call?

A **system call** is a **special request** from a user-space program to the OS to perform **privileged operations**.

✓ Why do we need system calls?

- User programs **cannot** directly:
 - Access hardware
 - Read/write files
 - Communicate with devices
- So they **ask the kernel** via system calls.

✓ How system calls work:

1. User process sets up parameters in registers/memory.
2. Executes a special instruction (e.g., `int 0x80` in Linux x86) to **trigger a trap**.
3. CPU switches to **kernel mode**.
4. The kernel executes the system call.
5. Returns result to the process.
6. CPU switches back to **user mode**.

✓ Common System Calls (examples):

- `read()` : Read data from a file/device.
- `write()` : Write data.
- `open()` : Open a file.
- `close()` : Close a file descriptor.
- `fork()` : Create a new process.
- `exec()` : Execute a new program.
- `exit()` : Terminate a process.

✓ Example:

In C:

```
write(1, "Hello\n", 6);
```

This code:

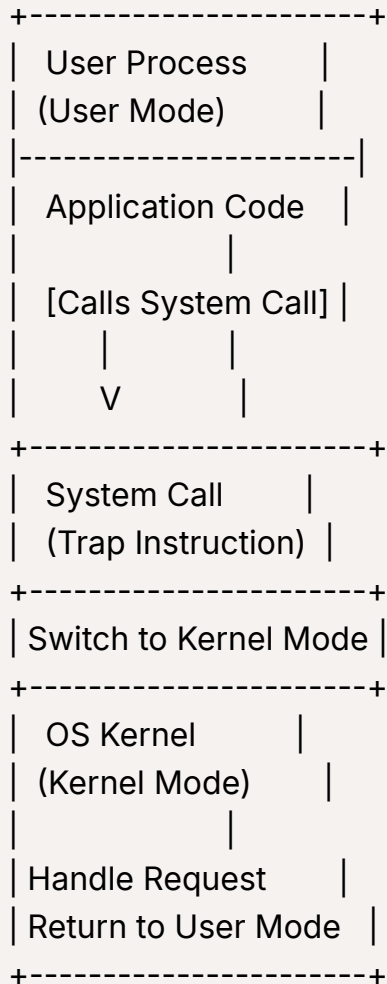
- Calls `write()` system call.
- Kernel writes to file descriptor 1 (stdout).

✓ MCQ Tip:

- "System calls are executed in which mode?" → Kernel Mode
 - "Which instruction is used to transfer control to the kernel?" → Trap or Software Interrupt
-

Diagram Recap

Here's a simple diagram to visualize everything:



Quick Summary Table

Concept	Description
User Space	Memory area where user processes run; limited access
Kernel Space	Memory area where OS runs; full access
User Mode	CPU mode with restricted privileges
Kernel Mode	CPU mode with full privileges

Concept	Description
Interrupt	Hardware/software signal to CPU to handle events
Trap	Software-generated interrupt (e.g., system call)
System Call	User program request to kernel to perform privileged operation

✓ Why does this separation exist?

- Protects the system.
- Prevents user programs from crashing or corrupting hardware.
- Ensures controlled access to resources.

Linux File System Basics

1 What is a File System?

A **file system** is the way an operating system **organizes, stores, and retrieves files** on a storage device (like HDD, SSD, USB).

✓ In Linux:

- Everything is treated as a **file** — this includes:
 - Regular files
 - Directories
 - Devices (/dev/sda, /dev/null)
 - Pipes, sockets
- The Linux file system is **hierarchical**, starting from a **single root**  **directory**.

2 Linux File System Structure (Hierarchy)

```

/
├── bin    → Essential binary commands (like ls, cp)

```

— boot	→ Boot loader files (kernel, grub)
— dev	→ Device files (hard disks, USB, terminals)
— etc	→ Configuration files (network, user accounts)
— home	→ User home directories (/home/user1)
— lib	→ Shared libraries required by binaries
— media	→ Mount point for external media (USBs, CDs)
— mnt	→ Temporary mount point
— opt	→ Optional/additional software
— proc	→ Virtual file system with process info
— root	→ Home directory of root user
— run	→ Runtime variable files
— sbin	→ System administration binaries (e.g. reboot)
— tmp	→ Temporary files
— usr	→ User programs (subdirectories: bin, lib, include)
— var	→ Variable data like logs, mail, spool

3 Types of Files in Linux

Linux supports **different types of files**, all represented within the file system:

Type	Description	Example
Regular File	Contains data, text, or program binaries	<code>file.txt</code> , <code>a.out</code>
Directory	A special file that holds other files	<code>/home</code> , <code>/etc</code>
Symbolic Link	Shortcut/alias to another file	<code>ln -s target link</code>
Character Device	Represents devices like keyboard, mouse	<code>/dev/tty</code> , <code>/dev/random</code>
Block Device	For devices that store data in blocks	<code>/dev/sda</code> , <code>/dev/sdb</code>
Named Pipe	Allows inter-process communication	<code>mkfifo mypipe</code>
Socket	Used for network communication	<code>/var/run/docker.sock</code>

4 Important Linux Directories

Directory	Purpose
/	Root directory of the system
/home	Contains personal folders for users
/etc	Configuration files (network, services)
/bin	Essential command binaries (used in single-user mode)
/sbin	System binaries for admin tasks
/var	Logs, spool files, and temporary changing data
/usr	Secondary hierarchy for user apps and libraries
/dev	Device files
/proc	Info about running processes and kernel parameters

5 Path Types

✓ Absolute Path:

- Starts from the **root** /.
- Example: /home/user1/Documents/file.txt

✓ Relative Path:

- Starts from the **current working directory**.
- Example: If you're in /home/user1, then Documents/file.txt is a relative path.

6 File Permissions and Ownership

✓ File Structure Format (using ls -l):

```
-rwxr-xr-- 1 user1 usergroup 1234 Jul 8 10:00 script.sh
```

Breakdown:

Field	Meaning
-	File type (- for regular, d for directory)

Field	Meaning
<code>rwX</code>	Owner permissions (read, write, execute)
<code>r-X</code>	Group permissions
<code>r--</code>	Other (world) permissions
<code>user1</code>	Owner
<code>usergroup</code>	Group

7 Mounting in Linux

- Linux **mounts file systems** into directories.
- You can add storage (e.g., USB) and **mount it** to a folder.

Commands:

- `mount /dev/sdb1 /mnt/usb`
- `umount /mnt/usb`

✓ The mount point becomes the **access path** to the device.

MCQ Cheat Sheet (Exam-Focused Points)

- ✓ Linux file system starts from? → `/` (root)
- ✓ Command to show current directory? → `pwd`
- ✓ Command to list all files including hidden? → `ls -a`
- ✓ Special file for hardware devices? → Block or Character file
- ✓ What type is `/dev/sda`? → Block device
- ✓ User home directory stored in? → `/home`
- ✓ Symbolic link created using? → `ln -s target linkname`
- ✓ Directory for config files? → `/etc`
- ✓ Where are logs stored? → `/var/log`

✓ **What does `rw xr-xr--` mean?** → Permissions: owner (all), group (read, execute), others (read only)

Session 2: Linux Commands, Redirection, Pipes, and File Permissions

1 Essential Commands for Files & Directories

Below are **frequently used commands**, grouped by what they do.

Working with Files

Command	Description	Example
<code>touch</code>	Create an empty file	<code>touch myfile.txt</code>
<code>cat</code>	View file content	<code>cat file.txt</code>
<code>more</code> / <code>less</code>	View file content page by page	<code>less file.txt</code>
<code>cp</code>	Copy file	<code>cp source.txt dest.txt</code>
<code>mv</code>	Move/rename file	<code>mv oldname.txt newname.txt</code>
<code>rm</code>	Remove (delete) file	<code>rm unwanted.txt</code>
<code>file</code>	Identify file type	<code>file myfile.txt</code>
<code>wc</code>	Count lines, words, characters	<code>wc myfile.txt</code>
<code>head</code>	Show first 10 lines	<code>head file.txt</code>
<code>tail</code>	Show last 10 lines	<code>tail file.txt</code>

Working with Directories

Command	Description	Example
<code>pwd</code>	Print current directory	<code>pwd</code>
<code>cd</code>	Change directory	<code>cd /home/user</code>

Command	Description	Example
<code>ls</code>	List files	<code>ls -l</code>
<code>mkdir</code>	Create directory	<code>mkdir newfolder</code>
<code>rmdir</code>	Remove empty directory	<code>rmdir oldfolder</code>
<code>rm -r</code>	Remove directory with content	<code>rm -r folder</code>

✓ Tip for MCQs:

- Command to create file? → `touch`
- Remove directory recursively? → `rm -r`

2 Other Basic Commands

Command	Purpose	Example
<code>echo</code>	Print text to terminal	<code>echo Hello World</code>
<code>date</code>	Show current date and time	<code>date</code>
<code>cal</code>	Display calendar	<code>cal</code>
<code>whoami</code>	Show current username	<code>whoami</code>
<code>uname</code>	Show system information	<code>uname -a</code>
<code>history</code>	Show command history	<code>history</code>
<code>clear</code>	Clear the screen	<code>clear</code>
<code>man</code>	Show manual page for a command	<code>man ls</code>

3 Operators: Redirection and Pipe

These **operators** are what make Linux powerful!

✨ Redirection Operators

Redirection sends input/output of commands to **files or devices** instead of the screen.

✓ Output Redirection

- `>` : Redirect output, **overwrite file**
 - `ls > filelist.txt`
- `>>` : Redirect output, **append to file**
 - `echo "New line" >> notes.txt`

✓ Input Redirection

- `<` : Read input from a file
 - `wc < file.txt` (counts lines/words/characters of file.txt)

✓ Example:

```
echo "Hello Linux" > myfile.txt # creates/overwrites
echo "Another line" >> myfile.txt # appends
```

✨ Pipe Operator

- The pipe `|` **connects output of one command to input of another.**

✓ Example:

```
ls -l | grep ".txt"
```

This shows only `.txt` files.

✓ Another example:

```
ps aux | less
```

This displays processes page by page.

✓ MCQ Tip:

- **What operator combines commands?** → Pipe (`|`)
- **Which operator appends output?** → `>>`

4 File Permissions

Linux is a **multi-user system**, so **permissions** control who can read, write, or execute files.

✨ Permission Categories

There are **three types of users**:

- **Owner (u)**: User who owns the file.
- **Group (g)**: Group assigned to file.
- **Others (o)**: All other users.

And **three types of permissions**:

- **r** (read): View contents.
 - **w** (write): Modify contents.
 - **x** (execute): Run as a program.
-

✨ Viewing Permissions

Use `ls -l`:

```
-rwxr-xr--
```

Breakdown:

Field	Meaning
-	Regular file (<code>d</code> = directory)
<code>rwX</code>	Owner: read, write, execute
<code>r-X</code>	Group: read, execute
<code>r--</code>	Others: read only

✨ Changing Permissions with `chmod`

✓ Two ways to set permissions:

◆ Symbolic Method

Use letters:

- **u** : user
- **g** : group
- **o** : others
- **a** : all

Operators:

- **+** : add permission
- **-** : remove permission
- **=** : set exact permission

Examples:

```
chmod u+x script.sh    # Add execute to owner
chmod go-w file.txt     # Remove write for group & others
chmod a+r notes.txt     # Add read to everyone
```

◆ Numeric (Octal) Method

Each permission has a number:

- **r** = 4
- **w** = 2
- **x** = 1

Add them up for each category:

Permission	Number
rwX	7 (4+2+1)
rw-	6 (4+2)
r--	4

Example:

```
chmod 755 script.sh
```

Breakdown:

- 7: owner (rwx)
- 5: group (r-x)
- 5: others (r-x)

✅ **Another example:**

```
chmod 644 file.txt
```

- 6: rw-
- 4: r--
- 4: r--

✅ **MCQ Tip:**

- Which command changes permissions? → `chmod`
- Octal for `rwxr-xr-x` ? → `755`
- Command to remove execute permission for all? → `chmod a-x filename`

✨ **Changing Ownership with `chown`**

- Change owner or group:

```
chown newuser:newgroup file.txt
```

✅ **Practical example:**

```
chmod 600 private.txt
```

Owner can read/write, others have no permissions.

Quick Recap Table

Task	Command Example
List files	<code>ls -l</code>
Create file	<code>touch file.txt</code>
Delete file	<code>rm file.txt</code>
Create directory	<code>mkdir folder</code>
Remove directory	<code>rm -r folder</code>
View file	<code>cat file.txt</code>
Append to file	<code>echo "text" >> file.txt</code>
Redirect output	<code>ls > list.txt</code>
Pipe commands	<code>`ls</code>
Change permissions	<code>chmod 755 script.sh</code>
Change owner	<code>chown user:group file.txt</code>

Permissions Recap: chmod, chown, chgrp

1. `chmod`

Changes file permissions.

Modes:

- **Symbolic:**
 - `u` (user), `g` (group), `o` (others), `a` (all)
 - `+`, `,`, `=`

Examples:

```
chmod u+x script.sh    # Add execute to user
chmod go-w file.txt    # Remove write for group & others
```

```
chmod a=r file.txt    # Everyone read-only
```

- **Numeric (Octal):**

- r=4, w=2, x=1
- Sum per category.

Examples:

```
chmod 755 script.sh    # rwx for owner, rx for group/others  
chmod 644 file.txt     # rw for owner, r for group/others
```

✓ **MCQ Tip:**

- `chmod 777` gives all permissions to everyone.

✓ **2. `chown`**

Changes file owner and group.

Syntax:

```
chown owner:group file
```

Examples:

```
chown alice file.txt    # Change owner  
chown alice:developers file.txt # Change owner & group
```

✓ **3. `chgrp`**

Changes group only.

Syntax:

```
chgrp groupname file
```

2 Access Control Lists (ACL)

Standard permissions (user, group, others) are sometimes **not enough**.

ACLs allow **more fine-grained control**, like giving **specific permissions to individual users**.

✓ **Basic commands:**

◆ **View ACL:**

```
getfacl filename
```

◆ **Set ACL:**

```
setfacl -m u:username:permissions filename
```

Example:

```
setfacl -m u:bob:rw file.txt
```

Bob can read and write.

◆ **Remove ACL:**

```
setfacl -x u:bob file.txt
```

◆ **Default ACL for new files in directory:**

```
setfacl -d -m u:bob:rwx myfolder
```

✓ **MCQ Tip:**

- Command to view ACL? → `getfacl`
- Command to modify ACL? → `setfacl`

3 Network Commands

Let's briefly see each tool:

✓ 1. telnet

- Used to connect to remote systems over TCP.
- **Not secure (plaintext).**

Syntax:

```
telnet hostname port
```

Example:

```
telnet 192.168.1.10 23
```

✓ 2. ftp

- File Transfer Protocol (plaintext).

Commands:

```
ftp hostname
```

Then inside ftp:

```
get filename  # Download
put filename  # Upload
bye           # Exit
```

✓ 3. ssh

- Secure remote login (encrypted).

Syntax:

```
ssh user@hostname
```

Example:

```
ssh root@192.168.1.20
```

✓ 4. sftp

- Secure File Transfer (SSH-based).

Usage:

```
sftp user@hostname
```

Inside sftp:

```
get file.txt  # Download  
put file.txt  # Upload
```

✓ 5. finger

- Shows information about a user (name, login time).

Syntax:

```
finger username
```

Example:

```
finger bob
```

✓ MCQ Quick Reference:

Command	Use
telnet	Remote login (insecure)
ftp	File transfer (insecure)
ssh	Secure login
sftp	Secure file transfer
finger	User info lookup

4 System Variables (PS1, PS2, etc.)

Environment variables define behavior in your shell (Bash, etc.).

✓ Common prompt variables:

Variable	Purpose
PS1	Primary prompt string (what you see before your command). Default: <code>\u@\h:\w\\$</code>
PS2	Secondary prompt (for multiline commands). Default: <code>></code>
PATH	List of directories to search for commands
HOME	User's home directory
USER	Username

✓ Changing Prompt:

Example:

```
PS1="MyPrompt> "
```

Prompt changes immediately.

✓ Common PS1 codes:

- `\u`: Username
- `\h`: Hostname
- `\w`: Current directory
- `\d`: Date
- `\t`: Time
- `\#`: Command number

Example with codes:

```
PS1="\u@\h \w> "
```

Shows:


```
alice@server /home/alice>
```

✔ Set variables temporarily:

```
VAR=value
```

Example:

```
MYNAME="Alice"  
echo $MYNAME
```

✔ Export variable to make it available to child processes:

```
export MYNAME
```

✔ Set variables permanently:

- Add to `~/.bashrc` or `~/.bash_profile`

Example:

```
echo 'export PS1="\u@\h \w> "' >> ~/.bashrc  
source ~/.bashrc
```

✔ MCQ Quick Reference:

- **PS1 controls what?** → Primary prompt
- **Command to set PS1?** → `PS1="newprompt"`
- **Command to make variable available to subshells?** → `export VAR`

Summary Table

Task	Command Example
Change permissions	<code>chmod 755 file</code>
Change owner	<code>chown user file</code>
Change group	<code>chgrp group file</code>
View ACL	<code>getfacl file</code>
Set ACL	<code>setfacl -m u:bob:rw file</code>
SSH login	<code>ssh user@host</code>
SFTP login	<code>sftp user@host</code>
FTP login	<code>ftp host</code>
Telnet login	<code>telnet host</code>
View user info	<code>finger user</code>
Show prompt variable	<code>echo \$PS1</code>
Set prompt temporarily	<code>PS1="Prompt> "</code>
Make variable available globally	<code>export VAR=value</code>

Shell Programming in Linux (Detailed)

1 What is a Shell?

Definition:

A **shell** is a **command-line interface (CLI)** that acts as a **middleman between the user and the kernel**.

- It **accepts commands**, **interprets** them, and **passes them to the kernel**.
- Think of it like a **translator** between you and the Linux system.

Two Roles:

- **Command Interpreter** – takes user commands and executes them.
- **Programming Language** – allows writing scripts using control structures, variables, loops, etc.

✓ MCQ Tip:

- "Shell is a bridge between?" → User and Kernel
 - "Is shell a part of the kernel?" → No
-

2 Types of Shells in Linux

Linux offers several shell types. Each has its own features.

Shell Name	Command	Description
Bourne shell	sh	Original Unix shell
Bash (Bourne Again SHell)	bash	Most common; default on most Linux
C shell	csh	Syntax like C programming
Korn shell	ksh	Improved Bourne shell
Z shell	zsh	Very advanced, powerful scripting

✓ Check current shell:

```
echo $SHELL
```

✓ Switch shell temporarily:

```
bash    # start bash
csh     # start c shell
```

✓ MCQ Tip:

- Default shell in most Linux distros? → Bash
 - Command to check current shell? → `echo $SHELL`
-

3 Shell Variables

Shell variables are **used to store values** for later use.

✓ Types of Variables

Type	Example	Description
User-defined	<code>name="Alice"</code>	Created by the user
System-defined	<code>\$HOME</code> , <code>\$PATH</code> , <code>\$USER</code>	Predefined by system

✓ Creating and Using Variables

```
name="Alice"  
echo $name
```

Note:

- No spaces around `=`
- Use `$` to access the value

✓ Exporting Variables

To use a variable in child processes:

```
export name
```

✓ Special System Variables:

Variable	Purpose
<code>\$HOME</code>	Home directory
<code>\$USER</code>	Username
<code>\$PATH</code>	List of directories searched for commands
<code>\$PWD</code>	Present working directory
<code>\$SHELL</code>	Current shell

✓ MCQ Tip:

- To access a variable? → Use `$variablename`

- To make variable global? → Use `export`

4 Wildcard Symbols (Globbing)

Wildcards are used to match file/directory names.

Symbol	Meaning	Example
<code>*</code>	Matches zero or more characters	<code>*.txt</code> → All <code>.txt</code> files
<code>?</code>	Matches exactly one character	<code>f?le</code> → file, f1le
<code>[]</code>	Matches any one character inside	<code>file[12]</code> → file1, file2
<code>[!x]</code>	Matches anything except x	<code>file[!1]</code> → file2, file3

✓ MCQ Tip:

- Wildcard for any one character? → `?`
- Wildcard for any number of characters? →

5 Shell Metacharacters

Metacharacters are **symbols with special meaning** in the shell.

Symbol	Description
<code>></code>	Redirect output
<code><</code>	Redirect input
<code>>></code>	Append output
<code>`</code>	<code>`</code>
<code>;</code>	Separate multiple commands
<code>&&</code>	Run second command only if first succeeds
<code>`</code>	
<code>*</code> <code>?</code> <code>[]</code>	Wildcards
<code>\</code>	Escape character
<code>"</code> <code>'</code>	Quoting
<code>\$</code>	Access variables

✓ Example:

```
echo "Name: $USER" > user.txt  
ls *.sh | grep install
```

✓ MCQ Tip:

- Which symbol is used for piping? → `|`
- To run second command only if first fails? → `||`

6 Command Line Arguments (`$1` , `$2` , ...)

Used in **shell scripts** to pass input from the command line.

✓ Example Script: `greet.sh`

```
#!/bin/bash  
echo "Hello, $1!"
```

Run it:

```
bash greet.sh Alice
```

Output:

```
Hello, Alice!
```

✓ Special variables:

Symbol	Meaning
<code>\$0</code>	Script name
<code>\$1</code>	First argument
<code>\$2</code>	Second argument

Symbol	Meaning
<code>\$#</code>	Total number of arguments
<code>\$@</code>	All arguments (quoted separately)
<code>\$*</code>	All arguments (as single string)

✓ MCQ Tip:

- Which variable holds script name? → `$0`
- To access second argument? → `$2`

7 `read` and `echo` Commands

✓ `read` command:

Used to **get input from user**.

Syntax:

```
read variable_name
```

Example:

```
echo "Enter your name:"  
read name  
echo "Hello $name"
```

✓ `echo` command:

Used to **display output**.

Examples:

```
echo "Welcome to Linux"  
echo "User: $USER"
```

✓ Escape characters with `-e`:

```
echo -e "Line1\nLine2\tTabbed"
```

8 Shell Script Example (All Concepts Combined)

Create a file `info.sh`:

```
#!/bin/bash

echo "Enter your name:"
read name
echo "Welcome, $name"

echo "This script is: $0"
echo "Total arguments: $# "
echo "Arguments are: $@"
```

Run it:

```
bash info.sh hello world
```

Summary Chart

Topic	Example / Command
Current shell	<code>echo \$SHELL</code>
Variable declaration	<code>name="Linux"</code>
Access variable	<code>echo \$name</code>
Export variable	<code>export name</code>
Wildcard 	<code>ls *.txt</code>
Wildcard 	<code>ls file?.txt</code>
Meta pipe	<code>`ls</code>
Meta redirect	<code>echo Hello > file.txt</code>

Topic	Example / Command
Argument <code>\$1</code>	First argument passed to script
<code>read</code> input	<code>read username</code>
<code>echo</code> output	<code>echo "Hi \$username"</code>

Shell Programming: Decisions, Loops, Regex, Arithmetic

1 Decision Making in Shell

The shell provides multiple ways to make decisions:

1. if...else

Example:

```
#!/bin/bash
num=5

if [ $num -gt 0 ]
then
    echo "Positive"
else
    echo "Non-positive"
fi
```

Points to remember:

- Spaces inside `[]` are **mandatory**.
- Comparison operators (`gt` , `lt` , `eq` , `ne`).

2. Nested if...else

You can nest multiple `if` conditions:

Example:

```
#!/bin/bash
num=0

if [ $num -gt 0 ]
then
    echo "Positive"
elif [ $num -lt 0 ]
then
    echo "Negative"
else
    echo "Zero"
fi
```

✓ **elif** means **else if**.

✓ 3. **test** Command

test is another way to write conditions.

Example:

```
if test $num -eq 0
then
    echo "Zero"
fi
```

✓ Equivalent to:

```
if [ $num -eq 0 ]
```

✓ 4. **case...esac**

Used for **multiple choices**, like switch-case in C.:

Example:

```
#!/bin/bash

echo "Enter a number (1-3):"
read num

case $num in
  1)
    echo "One"
    ;;
  2)
    echo "Two"
    ;;
  3)
    echo "Three"
    ;;
  *)
    echo "Invalid choice"
    ;;
esac
```

2 Loops in Shell

1. while Loop

Repeats as long as **condition is true**.

Syntax:

Example:

```
#!/bin/bash
count=1

while [ $count -le 5 ]
do
```

```
echo "Count: $count"
count=$((count + 1))
done
```

✓ 2. until Loop

Repeats as long as **condition is false**.

Syntax:

Example:

```
#!/bin/bash
count=1

until [ $count -gt 5 ]
do
    echo "Count: $count"
    count=$((count + 1))
done
```

✓ Difference:

- `while` : runs **while condition true**.
- `until` : runs **while condition false**.

✓ 3. for Loop

Iterates over a list of items.

Syntax:

Example:

```
#!/bin/bash

for fruit in apple banana cherry
do
```

```
echo "Fruit: $fruit"
done
```

✓ **C-style for loop** (Bash only):

```
for ((i=1; i<=5; i++))
do
    echo "i: $i"
done
```

3 Regular Expressions

Regular expressions (regex) are **patterns to match text**.

✓ **Basic Regex in Shell (with `grep`)**

Pattern	Meaning
<code>^</code>	Start of line
<code>\$</code>	End of line
<code>.</code>	Any single character
<code>*</code>	Zero or more of previous character
<code>[...]</code>	Any one character in brackets
<code>[^...]</code>	Any one character NOT in brackets
<code>\</code>	Escape special character

Examples:

1 Match lines starting with "Error":

```
grep "^Error" logfile.txt
```

2 Match lines ending with ".sh":

```
grep "\.sh$" files.txt
```

3 Match lines containing any digit:

```
grep "[0-9]" data.txt
```

✓ MCQ Tip:

-  = start of line.
-  = end of line.
-  = any character.

4 Arithmetic Expressions

Shell arithmetic can be done using:

✓ **`$(())`**

Example:

```
a=5
b=3
sum=$((a + b))
echo "Sum: $sum"
```

✓ **`let` command**

Example:

```
let c=a*b
echo "Product: $c"
```

✓ **`expr`**

Note: Needs spaces and  for multiplication.

Example:

```
result=`expr $a \* $b`
echo "Result: $result"
```

✓ Operators:

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus
**	Exponentiation

Processes (Sessions 4 & 5)

🎯 1 What is a Process?

✓ Definition:

A **process** is:

- An **instance of a program in execution**.
- Contains **code + data + resources** (memory, registers, files).

✓ Examples:

- When you run `firefox`, it becomes a **process**.
- When you run `ls`, each invocation is a **separate process**.

✓ MCQ Tip:

- *Process = Program in execution.*

✓ Attributes of a Process:

- **PID** (Process ID) – unique identifier.
- **PPID** (Parent Process ID).
- User who started it.

- Resources (memory, CPU time).
 - State (running, sleeping, etc.).
-

2 Preemptive vs Non-Preemptive Processes

These terms relate to **CPU scheduling** and process management.

Preemptive Process

- The **OS can interrupt (preempt)** a running process to give CPU to another.
- Ensures **fairness and responsiveness**.
- Used in **modern multitasking OS**.

Examples:

- Linux
- Windows
- macOS

MCQ Tip:

- *Which OS uses preemptive multitasking?* → Linux
-

Non-Preemptive Process

- A running process **keeps CPU until it finishes or voluntarily yields**.
- Simpler but less responsive.

Examples:

- Early MS-DOS
- Some embedded systems

MCQ Difference Table:

Preemptive	Non-Preemptive
OS can interrupt process	Process runs till completion or block

Preemptive	Non-Preemptive
More responsive	Less responsive
Better for multitasking	Simpler implementation

3 Difference Between Process and Thread

These are **commonly confused terms**—very important for MCQ!

Process:

- Independent execution unit.
- Has **separate memory space**.
- Switching between processes (**context switching**) is **heavier**.

Thread:

- **Lightweight unit within a process**.
- Shares **same memory** with other threads in the process.
- Faster to create and switch.

Analogy:

- **Process** = Apartment in a building.
- **Thread** = Roommates in that apartment.

Difference Table:

Aspect	Process	Thread
Memory	Separate memory space	Shared memory space
Creation	Slower, more overhead	Faster, lightweight
Communication	Via IPC (pipes, signals)	Direct, since shared memory
Dependency	Independent	Interdependent (one crash can affect all)

MCQ Tip:

- *Which shares address space?* → Threads.
- *Which is heavier to create?* → Process.

4 Process Management in OS

Process Management involves:

Creation:

- Using `fork()` or `exec()` in Unix/Linux.
- A parent process creates child processes.

Scheduling:

- Deciding **which process gets CPU** (using preemptive algorithms like Round Robin, Priority Scheduling).

Execution:

- Loading process into memory.
- Managing CPU registers.

Suspension/Resumption:

- Processes can be paused (sleep, wait) or resumed.

Termination:

- Process exits normally or due to error.

Linux commands for process management:

Command	Purpose
<code>ps</code>	List processes
<code>top</code>	Show running processes live
<code>kill</code>	Terminate a process
<code>nice</code> / <code>renice</code>	Change process priority
<code>bg</code> / <code>fg</code>	Background/foreground process

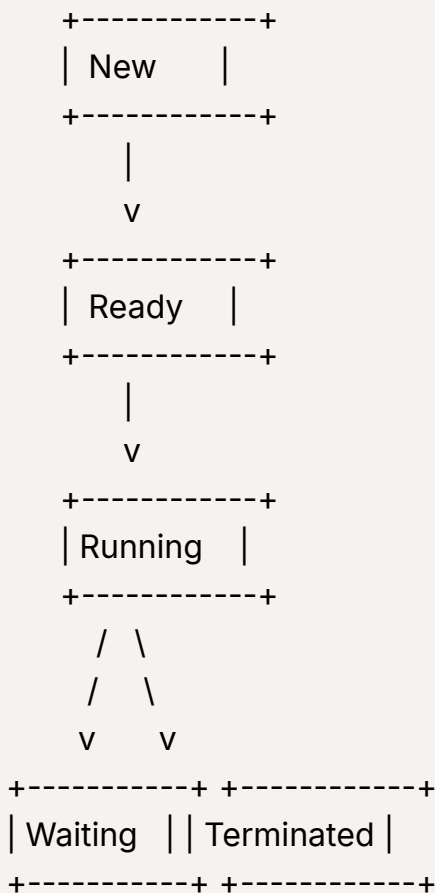
MCQ Tip:

- Which command lists processes? → `ps`
 - Command to kill a process? → `kill PID`
-

5 Process Life Cycle

Every process **goes through states**:

State Diagram:



States Explained:

State	Meaning
New	Process created but not yet ready
Ready	Process loaded in memory, waiting for CPU
Running	Currently executing on CPU
Waiting/Blocked	Waiting for I/O or event
Terminated	Process completed

✓ Transitions:

- **Running → Waiting:** Needs I/O.
- **Waiting → Ready:** I/O completed.
- **Running → Terminated:** Process exits.

✓ MCQ Tip:

- *State waiting for I/O?* → Blocked/Waiting.
- *State when process is executing?* → Running.

🟢 Quick Recap Table

Concept	Key Point
Process	Program in execution
Preemptive	OS can interrupt process
Non-preemptive	Process runs till it yields/completes
Process vs Thread	Thread is lightweight, shares memory
Process states	New, Ready, Running, Waiting, Terminated

🟢 Schedulers and Scheduling Algorithms

🎯 1 What Are Schedulers?

Schedulers are **parts of the OS that select which process to run next.**

Depending on **when** they do it, they are classified into three types:

✓ A. Long-Term Scheduler (Job Scheduler)

- Decides **which processes are admitted into the system (ready queue).**
- Controls **degree of multiprogramming** (how many processes are in memory).
- Frequency: Runs **rarely** (less frequently).

- Example: Batch systems decide which jobs to load.

✓ **MCQ Tip:**

- *Controls degree of multiprogramming?* → Long-term scheduler
-

✓ **B. Medium-Term Scheduler**

- **Swaps processes in/out of memory** (suspend and resume).
- Used in **swapping** for better memory management.
- Example: Suspends low-priority processes to free RAM.

✓ **MCQ Tip:**

- *Which scheduler handles swapping?* → Medium-term scheduler
-

✓ **C. Short-Term Scheduler (CPU Scheduler)**

- **Decides which process gets the CPU next.**
- Runs **very frequently** (milliseconds).
- Example: When time slice expires, picks next process.

✓ **MCQ Tip:**

- *Most frequently invoked scheduler?* → Short-term scheduler
-

Process Scheduling Algorithms

These algorithms decide **order of process execution**.

✓ **A. FCFS (First Come First Serve)**

- Processes are scheduled **in the order they arrive**.
- **Non-preemptive**.
- **Simple but poor performance** if long jobs arrive early.

✓ **Example:**

Process	Arrival	Burst Time
P1	0	5
P2	1	3
P3	2	8

Gantt Chart:

|P1|-----5-----|P2|--3--|P3|-----8-----|

Turnaround Time = Completion Time – Arrival Time

- P1: $(5-0)=5$
- P2: $(8-1)=7$
- P3: $(16-2)=14$

Average Turnaround Time = $(5+7+14)/3 = 8.67$

✓ MCQ Tip:

- *FCFS suffers from?* → Convoy effect (long job blocks short jobs)

✓ B. SJF (Shortest Job First)

- Executes **shortest burst time first**.
- **Non-preemptive**.
- **Optimal average turnaround time** if all burst times known.

✓ Example (same processes):

Process	Arrival	Burst
P1	0	5
P2	1	3
P3	2	8

Order:

- P1 (arrives first)

- P2 (3 units)
- P3 (8 units)

But since P2 arrives at 1, **P1 starts at 0**.

After P1 completes at 5, P2 and P3 are ready. SJF picks P2 (3 units).

Gantt Chart:

```
|P1|-----5-----|P2|--3--|P3|-----8-----|
```

Same as FCFS here because P1 starts before P2 arrives.

✓ SJF has **lower turnaround time** in many cases.

✓ **MCQ Tip:**

- *SJF advantage?* → Minimum average turnaround time.
- *SJF disadvantage?* → Starvation of long processes.

✓ C. Priority Scheduling

- Each process has **priority number**.
- **Lower number = higher priority** (conventionally).
- Can be **preemptive or non-preemptive**.

✓ Example:

Process	Burst	Priority
P1	10	3
P2	1	1
P3	2	2

Order:

- P2 (Priority 1)
- P3 (Priority 2)
- P1 (Priority 3)

Gantt Chart:

|P2|1|P3|2|P1|---10---

✓ MCQ Tip:

- Drawback? → Starvation of low-priority processes.

✓ D. Round Robin (RR)

- **Time-sharing system.**
- Each process gets a **fixed time quantum**.
- After quantum expires, next process runs.
- **Preemptive.**

✓ Example:

Process	Burst
P1	5
P2	4
P3	2

Time quantum = 2.

Gantt Chart:

P1 |2| P2 |2| P3 |2|
P1 |2| P2 |2|
P1 |1|

✓ Completion:

- P3 completes at 6.
- P2 at 8.
- P1 at 9.

✓ MCQ Tip:

- *Time quantum critical for?* → RR scheduling.
-

✓ E. Multi-Level Queue

- Processes **divided into queues** (foreground, background).
- Each queue has **own scheduling**.

✓ Example:

- Interactive processes → RR
 - Batch processes → FCFS
-

🟢 3 Belady's Anomaly

✓ Definition:

In **page replacement algorithms**, sometimes **more frames cause more page faults**, which is unexpected.

✓ Occurs in:

- **FIFO page replacement**

✓ Example:

- With 3 frames → 9 page faults
- With 4 frames → 10 page faults

✓ MCQ Tip:

- *Belady's anomaly occurs in?* → FIFO
 - *LRU shows Belady's anomaly?* → No
-

🟢 4 Example to Find Turnaround Time

Let's take this **FCFS vs SJF comparison example**:

✓ Processes:

Process	Arrival	Burst
P1	0	8
P2	1	4
P3	2	2

✓ FCFS:

Order:

P1 → P2 → P3

Gantt Chart:

|P1|--8--|P2|--4--|P3|--2--|

Process	Completion	Turnaround (CT - Arrival)
P1	8	8-0 = 8
P2	12	12-1 = 11
P3	14	14-2 = 12

Average Turnaround Time:

$$= (8 + 11 + 12) / 3 = 10.33$$

✓ SJF:

After P1 starts at 0 (since P2/P3 not yet arrived), when it finishes at 8:

P2 & P3 are ready.

SJF picks P3 (2 units), then P2.

Gantt Chart:

|P1|--8--|P3|2|P2|4|

Process	Completion	Turnaround
P1	8	8-0=8
P3	10	10-2=8

Process	Completion	Turnaround
P2	14	14-1=13

Average Turnaround:

$$= (8+8+13)/3 = 9.67$$

✅ **Better Performance:** SJF (lower turnaround).

🟢 Quick Recap Table

Scheduler	Role
Long-term	Admit jobs to system (control multiprogramming)
Medium-term	Swaps processes in/out
Short-term	Chooses process to run next

Algorithm	Features
FCFS	Simple, non-preemptive, poor performance with long jobs
SJF	Optimal average turnaround, can starve long jobs
Priority	Based on priority numbers, starvation possible
Round Robin	Time sharing, fair, quantum crucial
Multi-level	Different queues for different types

🟢 Process Creation and Related Concepts

🎯 1 Process Creation Using `fork()`

✅ **What is `fork()` ?**

- `fork()` is a **system call** that **creates a new process** by duplicating the calling process.
- The **new process is called the child**, and the **calling process is the parent**.

- The child gets a **copy of the parent's memory**, file descriptors, etc.

✓ Important Behavior:

- `fork()` returns:
 - `0` in the **child process**.
 - `>0` (child's PID) in the **parent process**.
 - `1` if error.

✓ MCQ Tip:

- *What does fork return in child? → `0`*

✓ Example in C:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
    } else if (pid == 0) {
        printf("Child Process: PID = %d\n", getpid());
    } else {
        printf("Parent Process: PID = %d, Child PID = %d\n", getpid(), pid);
    }

    return 0;
}
```

✓ Expected Output:

- Parent prints its PID and child PID.
- Child prints its own PID.

2 `exec()` System Calls

✓ What is `exec()` ?

- Replaces the current process image with a new program.
- After `exec()` , the **old program code is gone**.
- **Process ID remains the same**, but **code and memory change**.

✓ Types of `exec`:

- `execl()`
- `execp()`
- `execv()`
- `execvp()`

(They differ in how you specify the program and arguments.)

✓ Example: Parent runs `fork`, child runs `exec`

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child process
        execlp("ls", "ls", "-l", NULL);
        // If execlp fails
        printf("exec failed\n");
    } else {
        // Parent process
        wait(NULL);
        printf("Child finished\n");
    }
}
```

```
    return 0;
}
```

✓ What happens:

- `fork()` creates a child.
- Child replaces itself with `ls -l`.
- Parent waits.

✓ MCQ Tip:

- *Does exec create a new process?* → No.
 - *What does exec do?* → Replaces the process image.
-

3 `wait()` and `waitpid()`

✓ Purpose:

- Allows the **parent process to wait for child process termination**.
- Prevents **zombie processes**.

✓ `wait()` :

- Waits for **any child** to finish.
- Returns the PID of the terminated child.

✓ `waitpid()` :

- More control—waits for a **specific child**.
- Syntax:

```
waitpid(pid, &status, options);
```

- `pid` can be:
 - `1` (wait for any child)
 - Specific PID

- Other special values

✓ Example:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child process\n");
    } else {
        int status;
        waitpid(pid, &status, 0);
        printf("Parent: child exited\n");
    }

    return 0;
}
```

✓ MCQ Tip:

- Which call waits for child termination? → `wait()` or `waitpid()`

4 Parent and Child Processes

✓ Parent:

- Process that called `fork()`.

✓ Child:

- Duplicate of parent.
- Gets **new PID**.
- Parent and child execute **independently** after fork.

✓ Important:

- They have separate address spaces.
 - Modifying variables in child **does not affect parent**.
-

5 Orphan and Zombie Processes

✓ Orphan Process

- **Child continues running after parent terminates.**
- Automatically adopted by **init (PID 1)**.
- No harm to the system.

✓ Example:

Parent exits before child sleeps:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        sleep(5);
        printf("Child: parent gone, I'm orphan\n");
    } else {
        printf("Parent exits\n");
    }

    return 0;
}
```

✓ MCQ:

- *Who adopts orphan processes?* → init process (PID 1)
-

✓ Zombie Process

- **Child terminates but parent does not call** `wait()` .
- Entry remains in the **process table** (exit status).
- Appears in `ps` as `<defunct>` .

✓ Example:

Parent sleeps without `wait()` :

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child exiting\n");
    } else {
        sleep(10); // Parent does not call wait()
        printf("Parent exiting\n");
    }

    return 0;
}
```

✓ MCQ:

- *Zombie process is?* → Terminated process with no `wait()`
- *Danger?* → Accumulation can exhaust process table.

✓ Difference Table:

Orphan Process	Zombie Process
Parent terminates first	Child terminates first
Adopted by init	Entry remains in process table
Keeps running	Already dead

Orphan Process	Zombie Process
Not harmful	Can exhaust process slots if many

Quick Recap Table

Concept	Key Point
<code>fork()</code>	Creates new child process, returns PID or 0
<code>exec()</code>	Replaces process image with new program
<code>wait()</code>	Waits for any child to terminate
<code>waitpid()</code>	Waits for specific child
Orphan Process	Parent dies first, child adopted by init
Zombie Process	Child dies first, parent does not call wait, remains defunct

OS 2

🕒 Created @July 9, 2025 9:38 PM

Memory Management in Operating Systems

1 Different Types of Memories

✓ 1. Register

- Smallest, fastest memory.
- Inside CPU.
- Stores data currently being processed.

✓ 2. Cache

- Faster than RAM.
- Stores frequently used data to speed up access.

✓ 3. Main Memory (RAM)

- Temporary storage for programs and data being used.
- Volatile (data lost when power off).

✓ 4. Secondary Memory

- Permanent storage (HDD, SSD).
- Slower than RAM.

✓ 5. Virtual Memory

- Uses disk space to extend RAM capacity.
- Managed via paging/swapping.

✓ MCQ Tip:

- *Fastest memory?* → Register
 - *Volatile memory?* → RAM
-

2 What is the Need for Memory Management?

Memory Management = **Managing allocation, tracking, and freeing of memory in the system.**

✓ Why is it needed?

- So multiple processes can run without conflict.
- To **protect process memory spaces** from each other.
- To **maximize CPU utilization** by keeping memory full.
- To manage **swapping between RAM and disk (virtual memory)**.

✓ MCQ Tip:

- *Main purpose of memory management?*
 - Allocate memory, protect processes, optimize use.
-

3 Contiguous and Dynamic Allocation

✓ Contiguous Memory Allocation

- Each process occupies **one single continuous block** of memory.
- Simple to implement.
- **Problem:** Leads to **fragmentation** (free memory scattered in small holes).

✓ Example:

Memory:

```
+-----+-----+-----+-----+-----+
| OS | P1 | P2 |   |   |   |
+-----+-----+-----+-----+-----+
```

If P1 exits, a hole appears. New processes must fit exactly into holes.

✓ **MCQ Tip:**

- *Contiguous allocation drawback?* → Fragmentation.
-

✓ **Dynamic Memory Allocation**

- Allocates memory **as needed at runtime**.
- Processes can **request and free memory dynamically**.
- Managed using data structures (free lists).

✓ **Benefits:**

- Better utilization.
- Can handle variable-size processes.

✓ **Drawback:**

- More complex management.
- Still causes fragmentation.

✓ **MCQ Tip:**

- *Dynamic allocation uses?* → Free memory blocks managed dynamically.
-

4 Allocation Strategies (First Fit, Best Fit, Worst Fit)

When a process requests memory, the OS must **find a free hole large enough**.

These are the main strategies:

✓ **First Fit**

- **Finds the first hole big enough.**
- Allocates memory there.
- **Fast and simple.**

✓ **Example:**

Free blocks: [100 KB, 500 KB, 200 KB]

Request: 120 KB

- **First Fit:** Allocates 500 KB block.

✓ MCQ Tip:

- *First fit advantage?* → Faster allocation.
-

✓ Best Fit

- **Finds the smallest hole that fits.**
- Reduces wasted space (minimizes leftover).

✓ Example:

Free blocks: [100 KB, 500 KB, 200 KB]

Request: 120 KB

- **Best Fit:** Allocates 200 KB block (smallest suitable).

✓ Drawback:

- Slow (has to search all blocks).
- Can leave tiny unusable holes.

✓ MCQ Tip:

- *Best fit reduces?* → Wastage.
-

✓ Worst Fit

- Allocates the **largest available hole**.
- Idea: Leaves bigger holes for later allocations.

✓ Example:

Free blocks: [100 KB, 500 KB, 200 KB]

Request: 120 KB

- **Worst Fit:** Allocates 500 KB block.

✓ Drawback:

- Leads to larger leftover spaces that fragment over time.

✓ MCQ Tip:

- *Worst fit idea?* → Keep large holes for big processes.

✓ **Comparison Table**

Strategy	How it works	Advantage	Disadvantage
First Fit	First hole big enough	Fastest	Can cause fragmentation
Best Fit	Smallest suitable hole	Less wastage	Slow, small fragments
Worst Fit	Largest available hole	Keeps big free blocks	Poor utilization

Memory Management: Compaction & Fragmentation

1 Compaction

✓ Definition:

- **Compaction is the process of moving processes in memory to create one large contiguous free block.**

✓ Why is it needed?

- Over time, as processes are loaded and removed, **free memory becomes scattered in small holes.**
- Even if total free memory is enough, a large process **can't fit because it's not contiguous.**
- Compaction **eliminates external fragmentation** by shifting processes to one end.

✓ Visual Example:

Before Compaction:

```

+-----+-----+-----+-----+-----+-----+
| P1 | Free| P2 | Free| P3 | Free|
+-----+-----+-----+-----+-----+-----+

```

After Compaction:

```

+-----+-----+-----+       +-----+
| P1 | P2 | P3 | +   | One big Free Block |
+-----+-----+-----+       +-----+

```

✅ Important Notes:

- Only possible in systems with dynamic relocation (like modern OS).
- Consumes CPU time because it involves copying data.

✅ MCQ Tip:

- *What is compaction?* → Combining free memory spaces into one large block.
- *Which fragmentation does it solve?* → External fragmentation.

2 Internal Fragmentation

✅ Definition:

- Unused space within allocated memory blocks.
- Occurs when a process is allocated a **fixed-size partition or block** larger than it needs.
- The **extra space inside the block remains unused**.

✅ Example:

- Memory block size = 10 KB.
- Process needs = 7 KB.
- **3 KB wasted inside the allocated block.**

✅ Analogy:

- You book a hotel room with 4 beds but only use 2.

✓ Causes:

- Fixed-size partitioning.
- Paging (page size might be larger than actual data).

✓ MCQ Tip:

- *Fragmentation inside allocated block?* → Internal fragmentation.
- *Fixed-size partitioning leads to?* → Internal fragmentation.

3 External Fragmentation

✓ Definition:

- **Free memory is split into small, non-contiguous blocks.**
- Even though enough total free memory exists, **processes cannot be allocated because no single block is big enough.**

✓ Example:

Memory:

```
+-----+-----+-----+-----+-----+
| Used |Free |Used |Free |Used |
+-----+-----+-----+-----+-----+
```

- If process needs 15 KB:
 - Total free = 20 KB (5 KB + 15 KB).
 - But no single 15 KB block is available.

✓ Analogy:

- You have empty spots in parking, but no space big enough to park a bus.

✓ Causes:

- Variable-size allocations.
- Processes being loaded/unloaded dynamically.

✓ MCQ Tip:

- *Fragmentation between allocated blocks?* → External fragmentation.
- *Can external fragmentation be removed by compaction?* → Yes.

🟢 Quick Comparison Table

Aspect	Internal Fragmentation	External Fragmentation
Location	Inside allocated block	Between allocated blocks
Cause	Fixed-size partitions or pages	Variable-size allocations
Memory Wasted	Unused memory inside block	Free blocks scattered in memory
Removal?	Cannot be easily removed	Can be removed by compaction

✓ MCQ Tips Recap:

- *Compaction solves which fragmentation?* → External.
- *Fixed-size partitions lead to?* → Internal fragmentation.
- *Variable-size partitions lead to?* → External fragmentation.

🟢 Segmentation in Memory Management

🎯 1 What is Segmentation?

✓ Definition:

- **Segmentation is a memory management scheme** where:
 - Memory is divided into **variable-size segments**, each representing a **logical unit** (e.g., function, array, stack).
- Unlike paging (which splits memory into **fixed-size pages**), segmentation is **based on logical divisions**.

✓ Analogy:

Think of a book:

- Chapter 1 (Introduction)
- Chapter 2 (Theory)
- Chapter 3 (Examples)

Each chapter = **segment**.

✓ Why do we use segmentation?

- Matches **programmer's view of memory**:
 - Code segment
 - Data segment
 - Stack segment
- Provides **protection**: each segment can have separate access rights.

✓ MCQ Tip:

- *Segmentation divides memory based on?* → Logical divisions.
 - *Paging divides memory into?* → Fixed-size blocks.
-

2 Hardware Requirements for Segmentation

To implement segmentation, hardware must provide:

✓ Segment Table:

- Each process has a **segment table**.
- Contains **base address** and **limit (length)** of each segment.

✓ Segment Table Register:

- Holds the **starting address of the segment table in memory**.

✓ Segment Number and Offset:

- **Logical Address** = (Segment Number, Offset)

✓ Address Translation Hardware:

- Translates logical addresses to physical addresses.

- Ensures **offset < segment limit** (protection check).

✓ MCQ Tip:

- *What does each entry in segment table contain?* → Base and Limit.
 - *Logical address in segmentation?* → (Segment Number, Offset).
-

3 How Segmentation Works (Step by Step)

✓ Step 1: Logical Address

- Program generates a logical address (Segment Number, Offset).
 - Example: (2, 50)
 - Segment 2
 - Offset 50

✓ Step 2: Segment Table Lookup

- The OS looks up **segment 2** in the segment table.
 - Finds:
 - Base Address = 4000
 - Limit = 200

✓ Step 3: Protection Check

- **Is Offset < Limit?**
 - $50 < 200$ ✓ OK
- If not, **segmentation fault (invalid access)**.

✓ Step 4: Physical Address Calculation

- **Physical Address = Base + Offset**
 - $4000 + 50 = 4050$

✓ MCQ Tip:

- *What happens if offset exceeds limit?* → Trap (segmentation fault).

- *Physical address formula?* → Base + Offset.
-

4 Segmentation Table and Interpretation

✓ Structure of Segmentation Table Entry:

Segment Number	Base Address	Limit
0	1000	300
1	5000	600
2	4000	200
3	8000	500

✓ Example:

Logical Address = (2, 150)

- Segment 2: Base=4000, Limit=200
- Offset=150 < 200 ✓ Valid
- Physical Address = 4000 + 150 = 4150

✓ Invalid Example:

Logical Address = (1, 700)

- Segment 1: Base=5000, Limit=600
- Offset=700 ✗
- **Trap: Segmentation Fault**

✓ MCQ Tip:

- *Each segment table entry has?* → Base and Limit.
 - *Segmentation allows?* → Variable-sized logical divisions.
-

Quick Recap Table

Aspect	Segmentation
Division Basis	Logical units (code, data, stack)
Size of Segments	Variable
Address Format	(Segment Number, Offset)
Table Entry	Base Address, Limit
Protection	Offset checked against Limit
Fragmentation Type	External fragmentation possible

Paging in Memory Management

1 What is Paging?

✓ Definition:

- **Paging is a memory management scheme that eliminates external fragmentation by dividing memory into fixed-size blocks.**

✓ How it works:

- **Logical memory (process address space)** is divided into **pages** (fixed-size).
- **Physical memory (RAM)** is divided into **frames** (same size as pages).
- Pages are mapped to frames.

✓ Why use paging?

- No external fragmentation.
- Simpler memory allocation.
- Efficient utilization of RAM.

✓ MCQ Tip:

- *Paging divides logical memory into?* → Pages.
- *Paging divides physical memory into?* → Frames.

✓ Analogy:

Imagine cutting a book into same-size pieces (pages) and placing them wherever there is space in a bookshelf (frames).

2 Hardware Required for Paging

Paging requires special hardware support:

✓ Page Table:

- **Data structure that maps page numbers to frame numbers.**
- One page table **per process**.

✓ Page Table Base Register (PTBR):

- Holds the **starting address of the page table in memory**.

✓ MMU (Memory Management Unit):

- Translates logical addresses to physical addresses using the page table.

✓ MCQ Tip:

- *Who does address translation? → MMU.*
 - *What stores page table start address? → PTBR.*
-

3 Address Translation in Paging

Logical Address:

- Consists of:
 - **Page Number (p)**
 - **Offset within page (d)**

✓ Example:

- Page size = 4 KB = 4096 bytes.
- Logical Address = 8196.

Translation Steps:

1. **Page Number (p):**

$$\begin{aligned} p &= \text{Logical Address} / \text{Page Size} \\ &= 8196 / 4096 \\ &= 2 \end{aligned}$$

2. Offset (d):

$$\begin{aligned} d &= \text{Logical Address} \% \text{Page Size} \\ &= 8196 \% 4096 \\ &= 4 \end{aligned}$$

Suppose:

- Page Table Entry for $p=2$ maps to Frame Number = 5.

1. Physical Address:

$$\begin{aligned} \text{Physical Address} &= (\text{Frame Number} * \text{Frame Size}) + \text{Offset} \\ &= (5 * 4096) + 4 \\ &= 20484 \end{aligned}$$

✓ MCQ Tip:

- *Physical address formula?* → $(\text{Frame} * \text{Frame Size}) + \text{Offset}$.

4 Page Table

✓ Structure:

- Each entry contains:
 - **Frame Number** (where the page is stored)
 - Protection bits (read/write/execute)
 - Present/absent bit (for virtual memory)

✓ Example Page Table:

Page Number	Frame Number
0	3
1	7
2	5
3	1

✓ Interpretation:

- Page 0 is stored in Frame 3.
- Page 2 is stored in Frame 5.

✓ MCQ Tip:

- *Page table maps?* → Pages to Frames.

5 Translation Lookaside Buffer (TLB)

✓ Problem:

- Accessing RAM twice for every memory reference:
 - First to get frame number (page table lookup).
 - Second to access data.

✓ Solution:

- Use a **Translation Lookaside Buffer (TLB)**:
 - A **small, high-speed cache in the MMU**.
 - Stores recent page-to-frame translations.

✓ How it works:

1. CPU generates a logical address.
2. TLB is checked:
 - **Hit:** Frame number found in TLB → Fast access.
 - **Miss:** Must look in page table in RAM → Slower access.
 - Update TLB with the new mapping.

3. Data fetched from physical memory.

✓ **MCQ Tip:**

- *TLB purpose?* → Speed up address translation.
- *TLB is?* → Associative memory.

✓ **TLB Example:**

Page Number	Frame Number
2	5
3	1
0	3

✓ If Page 2 requested and in TLB → **TLB hit** → Fast translation.

🟢 Quick Comparison: Paging vs Segmentation

Aspect	Paging	Segmentation
Division	Fixed-size pages	Variable-size segments
Fragmentation	No external, may have internal	External fragmentation possible
Address	(Page Number, Offset)	(Segment Number, Offset)
Table Entry	Frame Number	Base Address + Limit

🟢 Recap Table

Concept	Key Point
Paging	Divide memory into fixed-size pages
Page Table	Maps pages to frames
PTBR	Holds page table starting address
TLB	Speeds up translation
MMU	Performs translation automatically

Concept of Dirty Bit

What is a Dirty Bit?

- A **flag (1-bit indicator)** associated with a **memory page or cache block**.
- It **tracks whether the page has been modified (written to)** after being loaded into RAM.

Purpose:

- During **paging or cache write-back**, the OS needs to know:
 - If the page has been changed (dirty).
 - If it's unchanged (clean), so it doesn't have to be saved back to disk.

How it works:

1. Page loaded into RAM → Dirty Bit = 0 (clean).
2. Process writes to page → Dirty Bit set to 1.
3. When the page is evicted:
 - Dirty Bit = 1 → Must **write back to disk** to save changes.
 - Dirty Bit = 0 → Just discard it.

Analogy:

- A library book with a sticky note:
 - No changes? Just put it back.
 - Marked pages? Save the updates.

MCQ Tip:

- *Dirty bit purpose?* → To track modified pages needing write-back.
-

Shared Pages and Reentrant Code

Shared Pages

- Multiple processes can **share the same physical memory pages** to save space.
- Common example: Shared code segments (like standard libraries).

✓ Reentrant Code

- Also called **pure code**.
- Code that:
 - **Does not modify itself while executing.**
 - Can be **shared safely by multiple processes** without side effects.

✓ Properties of Reentrant Code:

- **Read-only instructions.**
- Each process has its **own copy of data**, but they **share the same code segment**.
- Example: `printf()` in the C library.

✓ Why is this useful?

- Saves memory.
- Faster loading.

✓ MCQ Tip:

- *Reentrant code is?* → Code that does not modify itself.
- *Can be shared among?* → Multiple processes.

✓ Example:

Imagine two processes calling the same function:

- **Code (instructions)** are shared.
- **Stacks and variables** are separate.

Throttling

✓ Definition:

- **Throttling means intentionally slowing down a process or operation to control resource usage.**

✓ **Purpose:**

- Prevent overloading CPU, memory, or network.
- Maintain **Quality of Service (QoS)**.
- Avoid overheating or contention.

✓ **Where is throttling used?**

- **CPU scheduling** (reducing process priority).
- **Network I/O** (limiting bandwidth).
- **Web servers** (limiting requests per user).

✓ **Analogy:**

- Like a traffic light controlling how many cars can go.

✓ **MCQ Tip:**

- *Throttling is used to?* → Limit resource usage.
-

I/O Management

✓ **Definition:**

- **I/O Management refers to the OS's responsibility to handle all Input/Output operations between devices and processes.**

✓ **Responsibilities:**

1. Device Communication

- Managing hardware interfaces (drivers).

2. Buffering

- Temporarily storing data during transfers.

3. Caching

- Keeping frequently accessed data in memory.

4. Spooling

- Queuing data for slow devices (e.g., printing).

5. Error Handling

- Detecting and recovering from device failures.

6. Device Scheduling

- Deciding which process gets I/O next.

✓ MCQ Tip:

- *Buffering is?* → Temporarily holding data to match speed differences.
- *Spooling example?* → Print jobs queued in disk files.

✓ Analogy:

- Think of the OS as a manager handling:
 - Device drivers (communication).
 - Queues (spooling).
 - Temporary shelves (buffering).
 - VIP cache (fast access).

Quick Recap Table

Concept	Key Point
Dirty Bit	Flag to mark if page was modified since load
Shared Pages	Same physical memory used by multiple processes
Reentrant Code	Code that can be safely shared (does not self-modify)
Throttling	Slowing down processes to control resource usage
I/O Management	Managing device communication, buffering, spooling, scheduling

Virtual Memory

1 What is Virtual Memory?

✓ Definition:

- **Virtual memory is a technique that allows processes to use more memory than physically available in RAM.**
- It uses **disk space (usually a swap file or swap partition)** to simulate extra RAM.

✓ How it works:

- Each process gets a **large, continuous virtual address space**.
- The OS and hardware **map virtual addresses to physical memory (RAM) and disk (swap space)**.
- Only parts of a process actually needed right now are kept in RAM. The rest are stored on disk.

✓ Analogy:

- Like a library:
 - You have a small desk (RAM).
 - Most books (data) stay on shelves (disk).
 - You bring only the books you're currently reading onto your desk.

✓ Benefits:

- Allows large programs to run on limited memory.
- Provides **isolation and protection** between processes.
- Simplifies memory allocation.

✓ MCQ Tip:

- *Virtual memory uses?* → Disk space to extend RAM.
- *Benefit of virtual memory?* → Larger logical address space than physical memory.

2 Demand Paging

✓ Definition:

- **A technique where pages are loaded into RAM only when they are actually needed (on demand).**
- Opposite of loading the entire process into memory at start.

✓ How it works:

1. Process starts execution.
2. When it tries to access a page **not in memory**, a **page fault** occurs.
3. The OS loads the page from disk into RAM.
4. The instruction restarts.

✓ Analogy:

- Like cooking:
 - You don't bring all ingredients to your kitchen in advance.
 - You fetch them only when needed.

✓ Benefits:

- Less memory usage.
- Faster process startup.

✓ MCQ Tip:

- *Demand paging loads pages?* → When they are referenced.
-

3 Page Faults

✓ Definition:

- **A page fault occurs when a process tries to access a page that is not currently in RAM.**

✓ Steps in a Page Fault:

1. CPU generates a logical address.
2. MMU checks the page table:

- If **valid bit = 0**, page not in memory → Page fault.
3. OS suspends the process.
 4. Finds a free frame in RAM (or evicts an old page).
 5. Loads the needed page from disk into RAM.
 6. Updates the page table (sets valid bit = 1).
 7. Restarts the process instruction.

✓ MCQ Tip:

- *Page fault means?* → Page not in physical memory.
 - *Does every memory access cause a page fault?* → No, only first access if page not loaded.
-

4 Page Replacement Algorithms

✓ Why do we need them?

- When RAM is full and a new page must be loaded, **the OS must choose which page to evict.**
- This is called **page replacement.**

✓ Main Algorithms:

✓ a) FIFO (First-In, First-Out)

- **Oldest loaded page is removed first.**

✓ Example:

Frames: 3

Page sequence: 1 2 3 1 4 5

Steps:

- Load 1 (miss), 2 (miss), 3 (miss)
- Next 1 (hit)
- 4 (evict 1), 5 (evict 2)

✓ Pros:

- Simple.

✓ Cons:

- May evict frequently used pages.

✓ MCQ Tip:

- *FIFO removes?* → Oldest page.
-

✓ b) LRU (Least Recently Used)

- **Evicts the page that has not been used for the longest time.**

✓ Analogy:

- If you haven't read a book for a week, you remove it to make space.

✓ How it's implemented:

- Timestamps or stack.

✓ MCQ Tip:

- *LRU removes?* → Least recently used page.
-

✓ c) Optimal (OPT)

- **Evicts the page that will not be used for the longest time in the future.**
- Theoretically best but not implementable in practice (needs future knowledge).

✓ Example:

Frames: 3

Page sequence: 7 0 1 2 0 3 0 4 2

When page 2 is loaded, we look ahead:

- Page 7 used later? No.
- Page 0? Soon.

- Page 1? Later.
- Remove page with farthest use.

✓ MCQ Tip:

- *Optimal is used for?* → Benchmarking.
- *Practical?* → No.

✓ d) NRU, Clock, and Other Algorithms

✓ NRU (Not Recently Used):

- Uses reference and modified bits.

✓ Clock Algorithm:

- Circular buffer with reference bits.
- Similar to LRU approximation.

✓ MCQ Tip:

- *Clock algorithm is?* → Approximation of LRU.

✓ Comparison Table

Algorithm	How it works	Pros	Cons
FIFO	Evicts oldest page	Simple	Poor performance (Belady)
LRU	Evicts least recently used	Good approximation	Needs tracking usage
OPT	Evicts farthest future use	Best theoretically	Not implementable
Clock	Uses reference bits	Efficient practical	Approximation only

✓ Belady's Anomaly

- In FIFO, **more frames can sometimes lead to more page faults.**
- Does NOT happen in LRU or Optimal.

- **MCQ Tip:** *Belady's anomaly observed in?* → FIFO.

✓ Example MCQ

Page reference string: 1,2,3,4,1,2,5,1,2,3,4,5

Frames = 3

How many page faults with FIFO?

Answer: 9 (you can practice counting step by step).

🟢 Quick Recap Table

Concept	Key Point
Virtual Memory	Use disk to simulate more RAM
Demand Paging	Load pages only when needed
Page Fault	Access to a page not in RAM
FIFO	Remove oldest page
LRU	Remove least recently used
Optimal	Remove farthest used page (theoretical)
Belady's Anomaly	More frames → More faults in FIFO

🟢 Deadlock

🎯 What is Deadlock?

✓ Definition:

- A **deadlock** is a situation where a set of processes are all waiting indefinitely for events that can only be caused by each other.
- In other words:

- Each process **holds resources**.
- Each process **waits for resources held by others**.
- **None can proceed**.

✓ **Analogy:**

- Think of **four cars at a 4-way stop**—each waiting for the other to move first. No one moves.

✓ **MCQ Tip:**

- *Deadlock means?* → Processes waiting forever for resources held by each other.
-

🎯 **Necessary Conditions of Deadlock (Coffman Conditions)**

✓ **Four conditions must hold simultaneously:**

1 **Mutual Exclusion**

- Resources **cannot be shared** (only one process at a time).
- Example: Printer.

2 **Hold and Wait**

- Process **holds at least one resource** and **waits to acquire others**.

3 **No Preemption**

- Resources **cannot be forcibly taken away** from a process holding them.

4 **Circular Wait**

- A set of processes **{P1, P2, ..., Pn}** exists such that:
 - P1 waits for a resource held by P2,
 - P2 waits for P3,
 - ...,
 - Pn waits for P1.

✓ MCQ Tip:

- *How many conditions must hold for deadlock?* → All 4.
 - *Circular wait is?* → A closed chain of processes waiting on each other.
-

Deadlock Prevention

✓ Idea:

- **Ensure at least one of the four conditions never holds.**
- Break the conditions proactively.

✓ How to prevent?

- **Mutual Exclusion:** Make resources sharable if possible.
- **Hold and Wait:** Require processes to request **all resources at once**.
- **No Preemption:** Allow OS to **preempt resources**.
- **Circular Wait:** Impose **ordering on resources** and require requests in order.

✓ Example Prevention Technique:

- Always acquire locks in a pre-defined order to avoid cycles.

✓ MCQ Tip:

- *Deadlock prevention breaks which conditions?* → Any of the four.
-

Deadlock Avoidance

✓ Idea:

- Allow resources to be requested dynamically but **only grant requests if system stays in a safe state**.

✓ Algorithm Example:

- **Banker's Algorithm (Dijkstra)**
 - Think of the OS as a banker ensuring it always has enough resources for all customers to finish.

- Before granting a request, the OS checks if doing so keeps the system safe.
- If safe, grant.
- If unsafe, make process wait.

✓ **MCQ Tip:**

- *Banker's algorithm is used for?* → Deadlock avoidance.
 - *Safe state means?* → All processes can finish.
-

Semaphore

✓ **Definition:**

- A **synchronization tool (integer variable)** used to control access to shared resources.
- Semaphores can be **counting** or **binary (0/1)**.

✓ **Operations:**

1. **wait() / P():**

- Decrement the semaphore.
- If value becomes < 0 , the process blocks.

2. **signal() / V():**

- Increment the semaphore.
- If there are waiting processes, one is unblocked.

✓ **Example:**

- Semaphore = 1 (resource free).
- Process A: wait() → Semaphore=0 (locks resource).
- Process B: wait() → Semaphore=-1 (blocked).
- Process A: signal() → Semaphore=0, B wakes up.

✓ **MCQ Tip:**

- *Semaphore used for?* → Mutual exclusion and synchronization.
-

Mutex

✓ **Definition:**

- **A mutual exclusion lock that provides exclusive access to a resource.**
- Binary (locked/unlocked).
- Only the process that locked it can unlock it.

✓ **Difference from Semaphore:**

- **Semaphore can be counting or binary.**
- **Mutex is always binary and ownership-based.**

✓ **MCQ Tip:**

- *Mutex is?* → Mutual exclusion lock.
 - *Who can unlock mutex?* → The owner process.
-

Producer-Consumer Problem

✓ **Definition:**

- Classic synchronization problem:
 - **Producer** generates data and puts it in a buffer.
 - **Consumer** takes data from buffer.
 - Synchronization needed to:
 - Avoid buffer overflow (producer too fast).
 - Avoid buffer underflow (consumer too fast).

✓ **Solution:**

- Use semaphores:
 - **empty** (counts empty slots).

- **full** (counts filled slots).
- **mutex** (protects buffer access).

✓ Example Semaphores Initialization:

- `empty = N` (N = buffer size).
- `full = 0`.
- `mutex = 1`.

✓ Producer:

```
wait(empty);  
wait(mutex);  
// add item to buffer  
signal(mutex);  
signal(full);
```

✓ Consumer:

```
wait(full);  
wait(mutex);  
// remove item from buffer  
signal(mutex);  
signal(empty);
```

✓ MCQ Tip:

- *Producer-consumer solved with? → Semaphores.*



Deadlock vs. Starvation

✓ Deadlock:

- **All processes wait forever.**
- No progress at all.
- Circular wait.

✓ Starvation:

- **Some processes wait indefinitely** because others keep getting resources.
- System still progresses overall, but some processes are unfairly delayed.

✓ Example:

- Deadlock: Two processes each hold a lock the other needs.
- Starvation: A low-priority process never scheduled because higher-priority processes always run.

✓ MCQ Tip:

- *Deadlock vs starvation?*
 - Deadlock: System frozen.
 - Starvation: Some processes stuck, others proceed.

✓ Analogy:

- Deadlock = Four cars blocking each other in a gridlock.
- Starvation = One car never getting a chance to merge.

Quick Recap Table

Concept	Key Point
Deadlock	Processes waiting forever for each other
Coffman Conditions	Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait
Prevention	Break at least one condition
Avoidance	Banker's Algorithm, safe state checking
Semaphore	Integer variable for synchronization
Mutex	Binary lock owned by process
Producer-Consumer	Synchronize producer/consumer using semaphores
Deadlock vs Starvation	Deadlock: all stuck; Starvation: unfair waiting

Developing an Application in a Team

Issues Developers Face When Working in a Team

When multiple developers work together on the same project, they often face:

1. Code Conflicts

- Two or more developers **edit the same file or function**.
- Their changes may **clash**, requiring manual merging.

2. Overwriting Changes

- One person's code overwrites another's if updates aren't coordinated.

3. Version Confusion

- Which is the latest version?
- Who changed what and why?

4. Inconsistent Code

- Different coding styles, naming conventions, or approaches.

5. Lack of Visibility

- Team members don't know **who's working on which parts**.

6. Deployment Issues

- Moving code from development to production without a clear process can cause bugs.

7. Poor Collaboration

- Without tools, sharing and reviewing code is slow and error-prone.

MCQ Tip:

- *Key issue in team development?* → Code conflicts and overwriting.

Introduction to Code Versioning System

✓ Definition:

- A **code versioning system (or Version Control System—VCS)** is a tool that:
 - **Tracks changes** to source code over time.
 - **Records who made each change and why.**
 - **Allows collaboration without losing work.**

✓ Benefits:

- Never lose previous versions.
- Revert to any earlier version if needed.
- Merge changes from multiple developers.
- Audit history of the project.

✓ Analogy:

- Think of it as **Google Docs for code**—with a detailed history and the ability to merge edits.

✓ Examples:

- Git, Subversion, Mercurial, CVS.

✓ MCQ Tip:

- *Purpose of VCS?* → Track changes, collaborate safely, manage versions.
-

History of Code Versioning Systems

✓ Early Systems (1970s-1980s):

- **SCCS (Source Code Control System)** by AT&T.
- **RCS (Revision Control System)** improved SCCS, storing deltas (differences between files).

✓ Centralized VCS (CVCS):

- **CVS (Concurrent Versions System)**
 - Widely used in the 1990s.

- Central server stored all code.
- **Subversion (SVN)**
 - Improved CVS, easier branching, atomic commits.

✓ **Distributed VCS (DVCS):**

- **Git (2005, Linus Torvalds)**
 - Every developer has the **full repository locally**.
 - Faster operations, better branching.
- **Mercurial**
 - Another DVCS, simpler interface.

✓ **MCQ Tip:**

- *Git is?* → Distributed VCS.
 - *SVN is?* → Centralized VCS.
-

Different Tools Available for Versioning

✓ **Centralized Systems (CVCS):**

- **CVS**
- **Subversion (SVN)**
- **Perforce**

✓ **Distributed Systems (DVCS):**

- **Git**
- **Mercurial**
- **Bazaar**

✓ **Modern Collaboration Platforms:**

- **GitHub** (Git hosting + collaboration).
- **GitLab** (self-hosted Git + CI/CD).

- Bitbucket.

✓ **MCQ Tip:**

- *Example of DVCS? → Git.*
 - *Example of CVCS? → SVN.*
-

Software Development Workflow (with Version Control)

✓ **Typical Git-based Workflow:**

1 Clone Repository

- Copy the project to your local machine:

```
git clone https://github.com/org/repo.git
```

2 Create a Branch

- Work on a separate branch:

```
git checkout -b feature-branch
```

3 Make Changes

- Edit files, test locally.

4 Stage Changes

- Mark files for commit:

```
git add filename
```

5 Commit Changes

- Save changes with a message:

```
git commit -m "Implemented feature X"
```

6 Push Branch

- Upload your branch to the remote server:

```
git push origin feature-branch
```

7 Create Pull Request

- Request merging your changes into the main branch.

8 Code Review

- Teammates review and approve.

9 Merge

- Incorporate changes into main.

✓ Benefits:

- Each feature isolated in a branch.
- Code reviewed before merging.
- Complete history is tracked.

✓ Visual Diagram of Git Workflow:

```
[main]
|
+---[feature-branch]
    |
    (commits)
    |
    [Pull Request]
    |
    [Review + Merge]
```

Quick Recap Table

Concept	Key Point
Issues in Team Development	Conflicts, overwrites, confusion
Code Versioning System	Tool to track changes and manage collaboration
History of VCS	SCCS, RCS, CVS, SVN, Git
Centralized VCS Examples	CVS, SVN
Distributed VCS Examples	Git, Mercurial
Development Workflow	Clone → Branch → Commit → Push → Review → Merge

OS 3

🕒 Created @July 9, 2025 11:48 PM

Introduction to Git

✅ What is Git?

- **Git is a distributed version control system (DVCS).**
- Created by **Linus Torvalds** in 2005.
- Helps you **track changes, collaborate with teams, and manage different versions of code safely.**
- Each developer **has a complete copy of the entire repository history** locally.

✅ Why is Git so popular?

- Fast and lightweight.
- Supports **non-linear development** (branches).
- Allows **offline work.**
- Great community and tools (GitHub, GitLab).

✅ Analogy:

Git is like a time machine + collaboration platform for your code.

✅ MCQ Tip:

- *Git is?* → Distributed version control system.

Introduction to Git Repository and Git Structure

✓ Repository:

- A **repository (repo)** is a **directory project folder)** tracked by Git.
- Can be:
 - **Local repository:** On your machine.
 - **Remote repository:** On a server (e.g., GitHub).

✓ Git Directory Structure:

When you run `git init`, Git creates a hidden `.git` folder:

```
myproject/
├── .git/      ← Git metadata (history, config, branches)
├── file1.java
├── file2.js
└── README.md
```

✓ Important Areas Inside .git:

- `HEAD` : Points to your current branch.
- `objects/` : Stores all data and commits.
- `refs/` : Contains pointers to branches and tags.

✓ Three Important Areas in Git Workflow:

1 Working Directory

- Your actual files you are editing.

2 Staging Area (Index)

- Files you marked as ready to be committed.
- Think of it as **“things you want to pack.”**

3 Git Repository

- All committed snapshots permanently stored.

✓ Diagram:

[Working Directory] --git add → [Staging Area] --git commit → [Repository]

✓ **MCQ Tip:**

- *What is staging area?* → Place where changes are prepared before commit.

Adding Code to Git (Typical Workflow)

✓ **Step-by-Step Example:**

1 Initialize a Git repository:

```
git init
```

- Creates `.git/` folder.

2 Check status:

```
git status
```

- Shows untracked or modified files.

3 Add files to staging area:

```
git add filename
```

4 Commit staged files:

```
git commit -m "Initial commit"
```

- Creates a snapshot in repository history.

✓ **Analogy:**

- `git add` = Put files into the box you will send.
- `git commit` = Seal and label the box.

✓ MCQ Tip:

- *git add purpose?* → Stage changes.
 - *git commit purpose?* → Save snapshot.
-

Creating and Merging Different Git Branches

✓ What is a branch?

- A **pointer to a snapshot of your changes**.
- Lets you work on **features or fixes independently**.
- The **default branch** is usually called `master` or `main`.

✓ Why use branches?

- Isolate development.
- Try out experiments.
- Collaborate without interfering with the main code.

✓ Creating a branch:

```
git branch feature-xyz
```

✓ Switching to a branch:

```
git checkout feature-xyz
```

✓ Shortcut to create and switch:

```
git checkout -b feature-xyz
```

✓ Making changes:

- Edit files.

- `git add .`
- `git commit -m "Added new feature"`

✓ Merging a branch into main:

1. Switch to `main`:

```
git checkout main
```

1. Merge the feature branch:

```
git merge feature-xyz
```

✓ Deleting a merged branch:

```
git branch -d feature-xyz
```

✓ Diagram Example:

```
main:  A---B---C
      |
feature: D---E
```

After merge:

```
main:  A---B---C---F
      \ /
feature: D---E
```

(Commit F merges D and E into main.)

✓ MCQ Tip:

- *Default branch in Git?* → main or master.

- *git merge purpose?* → Combine changes from another branch.

GIT COMMANDS CHEAT SHEET

Repository Initialization

Command	Purpose
<code>git init</code>	Create a new Git repository
<code>git clone <url></code>	Copy an existing remote repository

Example:

```
git clone https://github.com/user/repo.git
```

Basic Workflow

Command	Purpose
<code>git status</code>	Show status of working directory
<code>git add <file></code>	Stage specific file
<code>git add .</code>	Stage all changed files
<code>git commit -m "message"</code>	Commit staged changes
<code>git log</code>	Show commit history
<code>git diff</code>	Show unstaged changes
<code>git diff --staged</code>	Show staged changes

Branching and Merging

Command	Purpose
<code>git branch</code>	List all branches

Command	Purpose
<code>git branch <branch-name></code>	Create a new branch
<code>git checkout <branch-name></code>	Switch to branch
<code>git checkout -b <branch-name></code>	Create and switch to new branch
<code>git merge <branch-name></code>	Merge branch into current branch
<code>git branch -d <branch-name></code>	Delete branch (safe)
<code>git branch -D <branch-name></code>	Force delete branch

Working with Remotes

Command	Purpose
<code>git remote -v</code>	List remotes
<code>git remote add origin <url></code>	Add remote repository
<code>git push -u origin <branch></code>	Push branch and set upstream
<code>git push</code>	Push committed changes
<code>git pull</code>	Fetch and merge from remote
<code>git fetch</code>	Fetch updates without merging

Inspecting Commits

Command	Purpose
<code>git show <commit></code>	Show details of a commit
<code>git log --oneline</code>	Compact commit history
<code>git log --graph</code>	Visual branch history
<code>git blame <file></code>	Show who changed each line

Undoing Changes

Command	Purpose
<code>git checkout -- <file></code>	Discard changes in working directory

Command	Purpose
<code>git reset <file></code>	Unstage file
<code>git reset --hard</code>	Reset working directory & staging area to last commit
<code>git revert <commit></code>	Create a new commit that undoes a previous one

Tags

Command	Purpose
<code>git tag</code>	List tags
<code>git tag <tagname></code>	Create lightweight tag
<code>git tag -a <tagname> -m "msg"</code>	Create annotated tag
<code>git push origin <tagname></code>	Push tag to remote

Helpful Shortcuts

Command	Purpose
<code>git stash</code>	Save uncommitted changes
<code>git stash apply</code>	Re-apply stashed changes
<code>git clean -f</code>	Remove untracked files
<code>git config --global user.name "Name"</code>	Set username
<code>git config --global user.email "email@example.com"</code>	Set email



Common Git Workflow Example

```
git clone https://github.com/user/repo.git
cd repo
git checkout -b feature-branch
# make changes
git add .
git commit -m "Added feature"
```



```
git push origin feature-branch  
# Create pull request on GitHub
```

✓ **TIP:**

If you ever forget a command, type:

```
git help
```

or for specific help:

```
git <command> --help
```

Introduction to Software Engineering

What is Software Engineering?

✓ **Definition:**

- **Software Engineering** is the **systematic application of engineering principles to develop, operate, maintain, and retire software.**
- It combines:
 - Technical skills
 - Management skills
 - Process discipline

✓ **Goals:**

- Deliver **high-quality software**
- Delivered **on time**
- Delivered **within budget**

✓ **MCQ Tip:**

- *Software Engineering applies?* → Engineering principles to software development.
-

Software Process

Definition:

- A **software process** is a **structured set of activities required to develop a software system**.

Main Activities:

1. **Specification**

- Define what the system should do.

2. **Design and Implementation**

- Create and build the software.

3. **Validation**

- Ensure the software does what is required.

4. **Evolution**

- Modify software to meet changing needs.

Analogy:

- Like **blueprints + construction + inspection + maintenance** of a building.
-

Software Process Model

Definition:

- A **software process model** is a **standardized representation of how to organize and carry out the activities of software development**.

Common Process Models:

1. **Waterfall Model**

- Linear sequence: Requirements → Design → Coding → Testing → Maintenance.

2. Iterative Model

- Develop incrementally, refining requirements in each iteration.

3. Spiral Model

- Repeated cycles focusing on risk analysis.

4. V-Model

- Validation and verification in parallel.

5. Agile

- Incremental, fast delivery, customer collaboration.

✓ MCQ Tip:

- *Waterfall is?* → Linear sequential model.
 - *Spiral emphasizes?* → Risk management.
-

Software Product

✓ Definition:

- A **software product** is the **final outcome** of the development process—a running program along with its documentation, installation files, and related artifacts.

✓ Examples:

- Web applications
- Mobile apps
- Desktop software

✓ Characteristics:

- **Functionality**
- **Reliability**

- **Usability**
- **Efficiency**
- **Maintainability**
- **Portability**

✓ **MCQ Tip:**

- *Software product includes?* → Code, documentation, configuration.
-

Importance of Software Engineering

✓ **Why is Software Engineering Important?**

1. Complexity Management

- Helps manage large, complex projects.

2. Quality Assurance

- Ensures reliable, error-free software.

3. Cost Reduction

- Early planning avoids rework.

4. Scalability

- Makes software easier to expand.

5. Faster Delivery

- Defined processes speed up development.

6. Customer Satisfaction

- Clear requirements and testing.

7. Team Collaboration

- Structured workflow supports teamwork.

✓ **MCQ Tip:**

- *Software Engineering helps to?* → Deliver quality software systematically.
-

Software Development Life Cycle (SDLC)

Definition:

- **SDLC** is the **step-by-step** process for developing software from idea to deployment and maintenance.

Phases of SDLC:

1. **Requirement Analysis**
2. **System Design**
3. **Implementation (Coding)**
4. **Testing**
5. **Deployment**
6. **Maintenance**

Popular SDLC Models:

- Waterfall
- Agile
- Spiral
- V-Model

MCQ Tip:

- *SDLC phases include?* → All steps from requirements to maintenance.
-

Requirements Engineering

Definition:

- **Requirements Engineering** is the process of **gathering, analyzing, documenting, and managing what the software should do.**

Objective:

- Create **clear, correct, and complete requirements**.

✓ **MCQ Tip:**

- *Purpose of requirements engineering?* → Understand and document user needs.
-

Types of Requirements

✓ **1. Functional Requirements**

- What the system **should do**.
- Example: "The system must generate monthly sales reports."

✓ **2. Non-Functional Requirements**

- **How** the system should be.
- Qualities or constraints:
 - Performance
 - Security
 - Usability
- Example: "The system should handle 10,000 transactions per second."

✓ **3. Domain Requirements**

- Requirements arising from the **application domain**.
- Example: Regulatory rules in banking.

✓ **MCQ Tip:**

- *Response time is?* → Non-functional requirement.
 - *Generating reports is?* → Functional requirement.
-

Steps Involved in Requirements Engineering

✓ **1. Feasibility Study**

- Is it technically and financially possible?

✓ 2. Requirements Elicitation

- Gather requirements from stakeholders.
 - Interviews
 - Surveys
 - Observation

✓ 3. Requirements Analysis

- Organize and prioritize requirements.

✓ 4. Requirements Specification

- Write clear, detailed requirements.
 - Typically in Software Requirements Specification (SRS) document.

✓ 5. Requirements Validation

- Check requirements for correctness and completeness.

✓ 6. Requirements Management

- Track and control changes to requirements.

✓ MCQ Tip:

- *Requirement validation ensures?* → Correctness of requirements.
 - *SRS is prepared in?* → Specification phase.
-

Requirement Analysis Modelling

✓ Definition:

- **Requirement Analysis Modelling** creates **visual models to understand requirements clearly**.

✓ Common Models:

1. Use Case Diagrams

- Show how users interact with the system.

2. Data Flow Diagrams (DFD)

- Show how data moves through the system.

3. Entity Relationship Diagrams (ERD)

- Show data entities and their relationships.

4. State Diagrams

- Show states and transitions.

✓ Purpose:

- Clarify requirements.
- Identify inconsistencies or missing details.

✓ MCQ Tip:

- *Use case diagram shows?* → Interactions between actors and system.

Quick Recap Table

Concept	Key Point
Software Engineering	Applying engineering principles to software development
Software Process	Structured activities for creating software
Software Process Model	Standard approach (Waterfall, Agile, etc.)
Software Product	Complete deliverable including code and documentation
SDLC	Phases from requirements to maintenance
Requirements Engineering	Process of gathering and managing requirements
Types of Requirements	Functional, Non-functional, Domain
Requirement Analysis Modelling	Use cases, DFD, ERD for clarity

AEEDDD Vedant bhosdike thik se padh.....

Design and Architectural Engineering

What is Software Design?

✓ Definition:

- **Software Design** is the process of **defining the architecture, components, interfaces, and data for a system** to satisfy specified requirements.

✓ Two Levels:

1. Architectural Design

- **High-level structure** of the system.
- Defines main modules and how they interact.

2. Detailed Design

- Defines **internal logic** of each module.
-

Characteristics of Good Design

✓ A good design has:

1 Correctness

- Satisfies all requirements.

2 Understandability

- Easy to read, review, and maintain.

3 Efficiency

- Optimizes resources (memory, CPU).

4 Modularity

- Divided into independent modules.

5 Reusability

- Components can be reused elsewhere.

6 Scalability

- Can grow without major redesign.

✓ MCQ Tip:

- *A good design should be?* → Correct, modular, efficient.
-

Function-Oriented vs Object-Oriented Systems

✓ Function-Oriented Design

- System organized around **functions and procedures**.
- Example: **Structured Design**, DFDs.
- Focus: **Data flow and transformations**.

✓ Object-Oriented Design

- System organized around **objects (data + behavior)**.
- Example: Classes and Objects.
- Focus: **Encapsulation, inheritance, polymorphism**.

✓ Comparison Table:

Aspect	Function-Oriented	Object-Oriented
Organization	Functions/Procedures	Objects/Classes
Emphasis	Actions/Processes	Data and Behavior
Reusability	Less	High
Example Diagram	Data Flow Diagram (DFD)	UML Class Diagram

✓ MCQ Tip:

- *Object-oriented emphasizes?* → Objects combining data and behavior.
-

Modularity, Cohesion, Coupling, Layering

✓ Modularity

- Dividing system into **independent modules**.
- Easier to **develop, test, and maintain**.

✓ Cohesion

- **Degree to which elements inside a module belong together.**

- **High cohesion = GOOD.**

✓ **Types of Cohesion** (from worst to best):

- Coincidental
- Logical
- Temporal
- Procedural
- Communicational
- **Functional (BEST)**

✓ **Coupling**

- **Degree to which modules depend on each other.**
- **Low coupling = GOOD.**

✓ **Types of Coupling** (from tight to loose):

- Content (worst)
- Common
- Control
- Stamp
- Data (best)

✓ **Layering**

- Dividing system into **layers (presentation, business logic, data)**.
- Each layer only interacts with adjacent layers.

✓ **MCQ Tip:**

- *High cohesion and low coupling are?* → Desired.
- *Functional cohesion means?* → Module does one task.

Design Models

✓ **Definition:**

- **Abstract representations of the system design.**

✓ **Examples:**

1. **Data Flow Diagrams (DFD)**

- Show data movement.

2. **Entity-Relationship Diagrams (ERD)**

- Show data entities and relationships.

3. **State Transition Diagrams**

- Show states and transitions.

4. **Class Diagrams**

- Show classes and their relationships.

5. **Sequence Diagrams**

- Show object interactions over time.

✓ **Purpose:**

- Help visualize and communicate design.

✓ **MCQ Tip:**

- *ERD used for?* → Data modeling.
-

UML (Unified Modeling Language)

✓ **Definition:**

- **Standard notation for modeling object-oriented systems.**

✓ **Important UML Diagrams:**

1. **Structural Diagrams**

- Class Diagram
- Component Diagram

- Deployment Diagram

2. Behavioral Diagrams

- Use Case Diagram
- Sequence Diagram
- Activity Diagram
- State Diagram

✓ Example Use Case Diagram:

```
[User] → (Login)  
[User] → (View Account)
```

✓ MCQ Tip:

- *UML stands for?* → Unified Modeling Language.

Coding

Programming Principles

✓ 1. Simplicity

- Write **simple, clear code**.

✓ 2. DRY (Don't Repeat Yourself)

- Avoid duplicate code.

✓ 3. KISS (Keep It Simple, Stupid)

- No unnecessary complexity.

✓ 4. YAGNI (You Aren't Gonna Need It)

- Don't code features "just in case."

✓ 5. Defensive Programming

- Handle unexpected inputs gracefully.

✓ 6. Code for Readability

- Prioritize **clarity over cleverness**.

✓ MCQ Tip:

- *DRY principle promotes?* → Avoiding repetition.
-

Coding Conventions

✓ Definition:

- **Guidelines on code style and formatting.**

✓ Examples:

- Naming conventions (`camelCase` , `PascalCase`).
- Indentation (4 spaces).
- Bracing style.
- Commenting standards.

✓ Purpose:

- Improve **readability, maintainability, and team consistency**.

✓ MCQ Tip:

- *Coding conventions help?* → Consistency and readability.
-

Object-Oriented Analysis and Design (OOAD)

✓ Definition:

- **Approach to analyze and design a system by modeling it as a group of interacting objects.**

✓ Key Concepts:

1. **Class**

- Blueprint for objects.

2. **Object**

- Instance of a class.

3. **Encapsulation**

- Data + behavior bundled.

4. **Abstraction**

- Focus on essential features.

5. **Inheritance**

- Reuse code by extending classes.

6. **Polymorphism**

- Same operation behaves differently on different classes.

✅ **OOAD Process:**

1. **Object-Oriented Analysis**

- Identify **what** the system must do (requirements).
- Model with:
 - Use Case Diagrams
 - Class Diagrams

2. **Object-Oriented Design**

- Define **how** the system will do it.
- Design class relationships, interactions.

✅ **Advantages:**

- Reusability.
- Modularity.
- Easier maintenance.

✅ **MCQ Tip:**

- *OOAD focuses on?* → Objects and their interactions.

Quick Recap Table

Concept	Key Point
Good Design	Correct, modular, efficient
Modularity	Divide into independent parts
Cohesion	Strength of intra-module connections (high = good)
Coupling	Degree of inter-module dependency (low = good)
Function vs Object-Oriented	Functions vs Objects as primary organization
UML	Standard modeling language
Programming Principles	Simplicity, DRY, KISS, readability
Coding Conventions	Style and formatting rules
OOAD	Modeling with objects, encapsulation, inheritance

Introduction to Agile Development Model

Definition:

- **Agile Development** is a **flexible, iterative, and incremental approach** to software development.
- It emphasizes:
 - Rapid delivery of working software.
 - Collaboration between developers and customers.
 - Quick response to changing requirements.

Key Ideas:

- Develop **small parts of the system quickly**.
- Gather feedback early.
- Adapt plans frequently.

Analogy:

- Instead of building the whole car at once, Agile builds:
 - A skateboard → A bicycle → A scooter → A motorcycle → The car.
- Each step delivers something usable.

✓ **MCQ Tip:**

- *Agile emphasizes?* → Iterative development and customer collaboration.
-

Agile Development Components

✓ **Core Components of Agile:**

1 Iterative Development

- Work done in **short cycles** (iterations or sprints).

2 Incremental Delivery

- Each iteration delivers a **potentially shippable product increment**.

3 Cross-Functional Teams

- Developers, testers, and business people work **together closely**.

4 User Stories

- Requirements written from the **user's perspective**.
- E.g., "As a customer, I want to receive an email receipt."

5 Continuous Feedback

- Stakeholders review increments and provide feedback.

6 Daily Stand-ups

- Short daily meetings to discuss progress and obstacles.

7 Backlog

- List of all work items (features, bugs).

✓ **MCQ Tip:**

- *User Stories are part of?* → Agile requirements.
-

Benefits of Agile

Main Benefits:

1. Flexibility

- Easily adapt to changing requirements.

2. Faster Delivery

- Working software delivered **early and often**.

3. Improved Quality

- Frequent testing and review.

4. Customer Involvement

- Continuous feedback keeps product aligned with user needs.

5. Better Risk Management

- Problems discovered early.

MCQ Tip:

- *Agile helps in?* → Faster delivery and adapting to change.
-

Introduction to Different Tools Used for Agile Web Development

Popular Tools:

Tool	Purpose
Jira	Tracking user stories, tasks, sprints
Trello	Visual boards for task management
Asana	Project tracking and collaboration
Slack	Team communication
GitHub / GitLab	Code versioning and collaboration

Tool	Purpose
Confluence	Documentation and knowledge sharing
VersionOne	Agile project management

✓ Tool Categories:

- **Issue Tracking:** Jira, Trello
- **Communication:** Slack, Microsoft Teams
- **Version Control:** Git
- **Documentation:** Confluence, Notion

✓ MCQ Tip:

- *Jira is used for?* → Agile project tracking.
-

Scrum

✓ Definition:

- **Scrum** is a **framework for managing Agile projects**.

✓ Roles in Scrum:

1. Product Owner

- Defines the product backlog.
- Prioritizes requirements.

2. Scrum Master

- Facilitates the process.
- Removes obstacles.

3. Development Team

- Builds the product increment.

✓ Key Events:

- **Sprint**

- Fixed time-boxed iteration (usually 2–4 weeks).
- **Sprint Planning**
 - Decide what to build this sprint.
- **Daily Scrum (Stand-up)**
 - 15-min daily meeting.
- **Sprint Review**
 - Demonstrate completed work to stakeholders.
- **Sprint Retrospective**
 - Reflect on what went well and what can improve.

✓ **Artifacts:**

- **Product Backlog**
 - All work items.
- **Sprint Backlog**
 - Work selected for the sprint.
- **Increment**
 - Completed, shippable product piece.

✓ **MCQ Tip:**

- *Sprint duration?* → 2–4 weeks.
 - *Scrum Master role?* → Facilitator.
-

Extreme Programming (XP)

✓ **Definition:**

- **Extreme Programming (XP)** is an **Agile methodology** that emphasizes **technical excellence**.

✓ **Core Practices:**

1. Pair Programming

- Two developers work together on the same code.

2. Test-Driven Development (TDD)

- Write tests before writing code.

3. Continuous Integration

- Integrate code frequently.

4. Refactoring

- Continuously improve code structure.

5. Small Releases

- Deliver working software frequently.

6. Simple Design

- Keep design as simple as possible.

✅ XP Values:

- Communication
- Simplicity
- Feedback
- Courage
- Respect

✅ MCQ Tip:

- *XP emphasizes?* → Pair programming and TDD.
-

Quick Recap Table

Concept	Key Point
Agile Development	Iterative, incremental, customer-focused
Agile Components	Iterations, user stories, backlog, daily stand-ups

Concept	Key Point
Benefits	Flexibility, faster delivery, quality
Scrum	Roles: Product Owner, Scrum Master, Team
Extreme Programming (XP)	Pair programming, TDD, simple design
Agile Tools	Jira, Trello, GitHub, Slack

Introduction to Atlassian Jira

What is Jira?

- **Jira** is a **popular project management tool** developed by **Atlassian**.
- Widely used for:
 - **Issue tracking**
 - **Agile project management (Scrum and Kanban)**
 - **Task management**
- Teams use Jira to **plan, track, and release software**.

Key Capabilities:

- Create projects and sprints
- Manage backlogs
- Assign tasks to team members
- Track progress visually (boards and reports)

MCQ Tip:

- *Jira is used for?* → Agile project tracking and issue management.

Adding a Project in Jira

Steps to Add a New Project:

Log In

- Log into your Jira instance.

2 Go to Projects

- From the top navigation, click **Projects > Create Project**.

3 Choose Project Template

- **Scrum Software Development**
- **Kanban**
- **Bug Tracking**
- For Agile, pick **Scrum**.

4 Configure Project

- Enter:
 - **Project Name** (e.g., "Online Shopping App")
 - **Project Key** (auto-generated or custom)
- Choose **Project Lead** (default is you).

5 Create

- Click **Create** to set it up.

✓ Result:

- You have a **workspace with Backlog, Boards, and Reports** ready.

✓ Visual Example:

[Projects] → [Create Project] → [Scrum Template] → [Configure] → [Create]

Adding Tasks and Sub-Tasks

✓ Tasks in Jira = Issues

- An **Issue** can be:

- Epic (big feature)
- Story (functional requirement)
- Task (work item)
- Bug
- Sub-task (smaller part of a task)

✓ Steps to Add a Task:

1 Go to Backlog or Board

2 Click "Create"

3 Fill Details:

- **Issue Type:** Task
- **Summary:** Short title (e.g., "Design Login Page")
- **Description:** Details of the work
- **Assignee:** Who will do it
- **Priority:** (High, Medium, Low)

✓ Create Sub-Tasks:

- Open a Task
- Click **Create Sub-Task**
- Enter:
 - Summary
 - Description

✓ Example Hierarchy:

```

Task: Design Login Page
├── Sub-task: Create HTML mockup
└── Sub-task: Write CSS styles
  
```

✓ MCQ Tip:

- *Sub-tasks are used to?* → Break down tasks into smaller work units.
-

Create Sprints with Tasks

What is a Sprint?

- A **time-boxed period** (e.g., 2 weeks) during which team completes a set of tasks.

Steps to Create and Start a Sprint:

1 Go to Backlog

- You'll see:
 - Backlog Items
 - "Create Sprint" button

2 Click "Create Sprint"

- A new sprint section appears.

3 Add Tasks to Sprint

- Drag and drop Stories/Tasks into the sprint.

4 Start Sprint

- Click **Start Sprint**
- Set:
 - Sprint Name (e.g., Sprint 1)
 - Duration (e.g., 2 weeks)
 - Start and End Date

During the Sprint:

- Move tasks across statuses:
 - To Do
 - In Progress

- Done

✓ MCQ Tip:

- *Sprint duration?* → Typically 1–4 weeks.
-

Case Study: Developing a Web Application Using Agile Methodology

Let's walk through a **practical example** of applying Agile + Jira.

✓ Scenario:

Develop an E-commerce Web Application to allow customers to browse products, add to cart, and checkout.

✓ Step 1: Create the Project

- In Jira, create a **Scrum project** called **E-Commerce Website**.
-

✓ Step 2: Define the Product Backlog

- Create **Epics**:
 - User Management
 - Product Catalog
 - Shopping Cart
 - Payment Integration
 - Break Epics into **User Stories / Tasks**:
 - *As a user, I want to register an account.*
 - *As a user, I want to search products.*
 - *As a user, I want to checkout securely.*
-

✓ Step 3: Prioritize Backlog

- Product Owner orders items by importance:
 1. User Registration
 2. Product Browsing
 3. Shopping Cart
 4. Checkout
-

✓ **Step 4: Plan Sprint 1**

- Duration: **2 weeks**
 - Sprint Backlog Items:
 - User Registration Page
 - Login Functionality
 - Database setup for Users
-

✓ **Step 5: Daily Scrum**

- Every day, team meets for 15 minutes:
 - What did you do yesterday?
 - What will you do today?
 - Any blockers?
-

✓ **Step 6: Development**

- Tasks assigned in Jira:
 - *Task:* Design registration page UI
 - *Sub-task:* Create HTML/CSS
 - *Task:* Implement user login backend
 - *Sub-task:* Write authentication logic
-

✓ **Step 7: Sprint Review**

- Demo working registration and login to stakeholders.
-

✓ Step 8: Sprint Retrospective

- Team discusses:
 - What went well?
 - What can improve next sprint?

✓ Step 9: Next Sprint

- Plan for Product Browsing and Search.

✓ Outcome:

- Every sprint delivers **incremental functionality**.
- Jira tracks progress, assignments, and backlog.

Quick Recap Table

Concept	Key Point
Jira	Tool for Agile project management
Project	Workspace for issues and sprints
Task	Work item assigned to team member
Sub-task	Smaller part of a task
Sprint	Time-boxed iteration of work
Scrum	Framework with roles and ceremonies
Case Study	E-commerce app built incrementally in sprints

Introduction to DevOps

✓ Definition:

- **DevOps** is a **combination of Development (Dev) and Operations (Ops)**.
- It is a **culture, set of practices, and tools** to:
 - Automate and integrate software development and IT operations.

- **Shorten the development life cycle.**
- Deliver high-quality software **continuously**.

✓ Main Objectives:

- **Continuous Integration and Continuous Deployment (CI/CD)**
- **Automation** (testing, deployment, monitoring)
- **Collaboration** (Dev + Ops teams working together)
- **Rapid feedback and iterations**

✓ Key Benefits:

- Faster delivery of features.
- More stable operating environments.
- Improved communication and collaboration.
- Early detection of problems.

✓ Analogy:

If traditional development is a relay race, DevOps is a soccer team where everyone works together, passing the ball continuously.

✓ MCQ Tip:

- *DevOps aims to?* → Integrate development and operations for faster delivery.

DevOps Ecosystem

✓ The **DevOps ecosystem** consists of **tools and practices** that cover the **entire software lifecycle**, including:

1. Version Control Systems

- To manage code.

- Example: **Git**, GitHub, GitLab
-

2. Continuous Integration (CI) Tools

- Automatically build and test code when changes are committed.
 - Example: **Jenkins**, Travis CI, CircleCI
-

3. Continuous Deployment/Delivery (CD) Tools

- Automatically deploy tested code to production.
 - Example: **Jenkins**, Bamboo, GitLab CI/CD
-

4. Configuration Management

- Automate server setup and configuration.
 - Example: **Ansible**, Puppet, Chef
-

5. Containerization and Orchestration

- Package applications into containers for consistent environments.
 - Example: **Docker** (containerization), Kubernetes (orchestration)
-

6. Monitoring and Logging

- Track performance and errors.
 - Example: **Prometheus**, Grafana, ELK Stack (Elasticsearch, Logstash, Kibana)
-

MCQ Tip:

- *Docker is used for?* → Containerization.
 - *Jenkins is mainly used for?* → Continuous Integration.
-

DevOps Phases

✓ The DevOps lifecycle has **7 main phases** (sometimes called the **DevOps infinity loop**):

1 Plan

- Define requirements.
 - Plan releases.
 - Tools: Jira, Trello.
-

2 Code

- Develop application.
 - Use version control.
 - Tools: Git, GitHub.
-

3 Build

- Compile code, run unit tests.
 - Tools: Maven, Gradle.
-

4 Test

- Automated testing.
 - Tools: Selenium, JUnit.
-

5 Release

- Deploy to staging or production.
 - Tools: Jenkins, Bamboo.
-

6 Deploy

- Deliver application to users.
 - Tools: Docker, Kubernetes.
-

7 Operate & Monitor

- Keep system running.

- Monitor performance.
- Tools: Nagios, Prometheus.

✓ **MCQ Tip:**

- *Which phase involves automated testing?* → Test.
- *Which phase uses Kubernetes?* → Deploy.

✓ **Visual Diagram (DevOps Infinity Loop):**

[Plan] → [Code] → [Build] → [Test] → [Release] → [Deploy] → [Operate & Monitor] → (loops back to Plan)

Introduction to Containerization

✓ **Definition:**

- **Containerization** packages an application and its dependencies into a **single lightweight unit called a container**.
- This ensures the application runs the **same everywhere**, regardless of the environment.

✓ **Difference Between VM and Container:**

Virtual Machine	Container
Heavyweight (GBs)	Lightweight (MBs)
Runs full OS	Shares host OS kernel
Slow to start	Starts in seconds
Example: VMware, VirtualBox	Example: Docker

✓ **Advantages:**

- Consistent environments.
- Faster deployment.
- Better resource utilization.

- Isolation of applications.

✓ Popular Tool:

- **Docker:** Industry-standard for creating and managing containers.

✓ MCQ Tip:

- *Docker is used for?* → Containerization.
- *Containers share?* → Host OS kernel.

✓ Simple Analogy:

A container is like a shipping container—whatever you put inside, it runs exactly the same whether on a ship, truck, or train.

Quick Recap Table

Concept	Key Points
DevOps	Combines Dev and Ops for fast, reliable delivery
DevOps Ecosystem	Tools: Git, Jenkins, Docker, Kubernetes, Ansible
DevOps Phases	Plan, Code, Build, Test, Release, Deploy, Operate
Containerization	Package apps in containers (e.g., Docker) for consistency

Introduction to Docker

✓ What is Docker?

- **Docker** is a **platform to develop, ship, and run applications in containers.**
- It lets you:

- Package your application and all dependencies (libraries, configs) in a single **container image**.
- Run the container consistently across **any environment** (laptop, server, cloud).

✓ Core Concepts:

1 Docker Image

- Read-only template with everything needed to run an application.
- Like a **blueprint**.

2 Docker Container

- A **running instance** of an image.
- Like the **actual object created from the blueprint**.

3 Docker Engine

- Core runtime that builds and runs containers.

4 Docker Hub

- Cloud-based repository to share Docker images.

✓ Analogy:

- Image = **Class**
- Container = **Object**

✓ MCQ Tip:

- *Docker is used for?* → Containerization.
- *Docker Image is?* → Read-only template.

Creating Docker Images using Dockerfile

✓ Dockerfile:

- A **text file with instructions** to build a Docker image automatically.

✓ Basic Workflow:

1 Write Dockerfile

- Define:
 - Base Image (e.g., `python:3.9`)
 - Files to copy
 - Commands to run
 - Exposed ports
 - Startup command

2 Build Image

- `docker build -t imagename .`

3 Run Container

- `docker run imagename`

✓ Example Dockerfile:

```
# 1. Use official Python base image
FROM python:3.9
```

```
# 2. Set working directory
WORKDIR /app
```

```
# 3. Copy application files
COPY . /app
```

```
# 4. Install dependencies
RUN pip install -r requirements.txt
```

```
# 5. Expose port
EXPOSE 5000
```

```
# 6. Command to run app
CMD ["python", "app.py"]
```

✓ Explanation of keywords:

Instruction	Purpose
FROM	Base image
WORKDIR	Working directory in container
COPY	Copy files into image
RUN	Execute commands during build
EXPOSE	Declare container port
CMD	Command that runs when container starts

✓ Build and Run Example:

```
# Build Docker image
docker build -t my-python-app .

# Run container
docker run -p 5000:5000 my-python-app
```

✓ MCQ Tip:

- Which instruction sets the base image? → FROM.
- docker build creates? → Docker image.

Docker Container Life Cycle

✓ A Docker container has the following **life cycle stages**:

1 Create

- Container is created from an image but **not running** yet.
- Command:

```
docker create imagename
```

2 Start

- Starts the created container.
- Command:

```
docker start containerID
```

3 Running

- Container is actively executing its processes.

4 Pause

- Temporarily suspends processes inside the container.
- Command:

```
docker pause containerID
```

5 Unpause

- Resumes paused container.
- Command:

```
docker unpause containerID
```

6 Stop

- Gracefully stops the container.
- Command:

```
docker stop containerID
```

7 Restart

- Stops and then starts the container again.
- Command:

```
docker restart containerID
```

8 Kill

- Immediately terminates the container.
- Command:

```
docker kill containerID
```

9 Remove

- Deletes the stopped container.
- Command:

```
docker rm containerID
```

✓ Life Cycle Flow:

Create → Start → Running

↘ ↘

Pause ↔ Unpause

↘

Stop → Remove

✓ MCQ Tip:

- *docker kill does?* → Forcefully stops container.
- *docker stop does?* → Graceful shutdown.
- *After stopping, container can be?* → Restarted or removed.

Quick Recap Table

Concept	Details
Docker Image	Read-only template
Docker Container	Running instance of an image
Dockerfile	Instructions to build image
Build Command	<code>docker build -t name .</code>
Run Command	<code>docker run name</code>
Life Cycle Commands	create, start, stop, restart, pause, remove

OS 4

🕒 Created @July 10, 2025 12:47 AM

aeeee baba vedant samaj nahi aata toh phir se padh par padh le reee

Introduction to YAML

✅ What is YAML?

- **YAML** stands for:

| YAML Ain't Markup Language

- It is a **human-readable data serialization format** often used for:
 - Configuration files
 - Data exchange between languages
 - Defining infrastructure (e.g., Kubernetes manifests, Docker Compose)

✅ Key Features:

- **Indentation-based hierarchy**
- Easy to read and write
- No brackets or closing tags

✅ Basic Syntax:

```
# Example: Configuration file
version: "3"
services:
  web:
    image: nginx
    ports:
      - "80:80"
```



```
db:
  image: mysql
  environment:
    MYSQL_ROOT_PASSWORD: example
```

✓ Rules:

- **Indentation matters** (always use spaces, NOT tabs)
- **Key-value pairs** separated by colon
- **Lists** indicated by dash ()
- **Comments** start with #

✓ MCQ Tip:

- *YAML is mainly used for?* → Configuration files
- *Indentation in YAML is?* → Mandatory

Introduction to Docker Swarm

✓ What is Docker Swarm?

- **Docker Swarm** is Docker's **native clustering and orchestration tool**.
- It lets you:
 - Group multiple Docker hosts (machines) into a **single virtual Docker engine**.
 - Deploy and manage containers across this cluster.
 - Handle load balancing, scaling, and fault tolerance.

✓ Key Concepts:

Term	Meaning
Node	A Docker Engine instance participating in the Swarm
Manager Node	Orchestrates the cluster (assigns tasks)
Worker Node	Runs containers (tasks)

Term	Meaning
Service	A distributed container across the Swarm
Task	A single container instance

✓ Benefits:

- Built-in with Docker
- Easy to set up
- Scales out container workloads

✓ Basic Workflow:

1. Initialize Swarm

```
docker swarm init
```

2. Add Nodes

- Workers join via token.

3. Deploy Service

```
docker service create --replicas 3 nginx
```

✓ MCQ Tip:

- *In Docker Swarm, a service is?* → A set of container tasks.

Introduction to Docker Stack

✓ What is Docker Stack?

- **Docker Stack** is a way to **deploy and manage a group of services defined in a YAML file** across a Swarm cluster.
- Think of it as a **multi-container application blueprint**.

✓ Use Case:

- Instead of deploying services one by one, define everything in a `docker-compose.yml` file and deploy the **whole stack**.

✓ Steps to Deploy a Stack:

1 Create a YAML file (e.g., `stack.yml`):

```
version: "3"
services:
  web:
    image: nginx
    ports:
      - "80:80"
  db:
    image: mysql
    environment:
      MYSQL_ROOT_PASSWORD: example
```

2 Deploy the stack:

```
docker stack deploy -c stack.yml mystack
```

3 View the stack:

```
docker stack ls
```

✓ MCQ Tip:

- *Docker Stack deploys?* → Multiple services as a single application.

Introduction to Kubernetes

✓ What is Kubernetes?

- **Kubernetes (K8s)** is a **powerful open-source container orchestration platform** developed by Google.
- It automates:

- Deployment
- Scaling
- Load balancing
- Healing (self-recovery) of containers

✅ Why Use Kubernetes?

- When you have **many containers**, Kubernetes:
 - Manages scheduling on cluster nodes
 - Automatically restarts failed containers
 - Scales up/down based on load
 - Performs rolling updates

✅ Core Concepts:

Term	Description
Cluster	Set of nodes managed by Kubernetes
Node	Worker machine (VM or physical)
Pod	The smallest deployable unit , one or more containers
Deployment	Declarative way to manage Pods and replicas
Service	Stable network endpoint to access Pods
ConfigMap	Stores configuration data
Ingress	HTTP(S) routing to services

✅ Basic Workflow:

1. Write YAML manifest (e.g., a Deployment)
2. Apply manifest:

```
kubectl apply -f deployment.yaml
```

3. Kubernetes schedules Pods on Nodes.

✅ Example YAML Deployment:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.19
          ports:
            - containerPort: 80

```

✓ MCQ Tip:

- *Pod in Kubernetes is?* → The smallest deployable unit.
- *Kubernetes developed by?* → Google.

Quick Recap Table

Concept	Key Point
YAML	Human-readable config format, indentation-based
Docker Swarm	Native clustering/orchestration in Docker
Docker Stack	YAML-defined multi-service application deployed to Swarm
Kubernetes	Advanced orchestration platform for containers

Absolutely—let's break this down into **three clear topics** with step-by-step guidance:

✓ **Creating a Kubernetes Cluster**

✓ **Creating a Service in Kubernetes**

✓ **Deploying an Application Using the Kubernetes Dashboard**

I'll explain each in *exam-focused, clear, and practical language*.

1 Creating a Kubernetes Cluster

What is a Kubernetes Cluster?

- A **cluster** = **Master (Control Plane)** + **Worker Nodes**
- Master node manages:
 - Scheduling
 - Scaling
 - Monitoring
- Worker nodes run **Pods** (containers).

✓ **Ways to Create a Cluster:**

There are many tools to set it up:

a) Minikube (For Local Testing)

- **Minikube** runs Kubernetes locally inside a VM or container.
- Great for practice and development.

✓ **Steps:**

1 Install Minikube (you can download from minikube.sigs.k8s.io)

2 Start Minikube:

```
minikube start
```

This creates a single-node cluster locally.

3 Verify the Cluster:

```
kubectl get nodes
```

You should see one node in "Ready" state.

b) kubeadm (Production / Multi-Node)

- **kubeadm** bootstraps Kubernetes clusters on VMs or physical machines.

✓ Basic Steps:

1 Install Docker + kubeadm + kubelet on all machines.

2 On master node:

```
kubeadm init
```

This outputs a **join token**.

3 On worker nodes:

```
kubeadm join <master-ip>:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash>
```

4 Set up kubectl on the master:

```
mkdir -p $HOME/.kube  
cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
```

5 Install a pod network (e.g., Flannel):

```
kubectl apply -f https://raw.githubusercontent.com/coreos/flannel/master/Documentation/kube-flannel.yml
```

✓ Exam Tip:

- *Minikube is used for?* → Local clusters.
- *kubeadm is used for?* → Setting up production clusters.

2 Creating a Service in Kubernetes

Why Services?

- Pods have dynamic IPs.
- **Services give stable access to Pods.**

Types of Services:

Type	Purpose
ClusterIP	Internal cluster access
NodePort	Exposes service on host ports
LoadBalancer	External access with load balancer
ExternalName	Maps service to external DNS name

Example: Creating a Service

Suppose you have a **Deployment** running Nginx Pods.

Here's a YAML to create a **NodePort Service**:

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30036
```

Steps:

1 Save the YAML to `service.yaml`.

2 Create the service:

```
kubectl apply -f service.yaml
```

3 Verify:

```
kubectl get service
```

You'll see an external port you can use to access the app.

✓ **Exam Tip:**

- *ClusterIP* is default.
- *NodePort* exposes service on node IP + port.

3 Deploying an Application Using Kubernetes Dashboard

✓ **What is Kubernetes Dashboard?**

- A **web-based UI** to:
 - Deploy applications
 - Monitor resources
 - Manage cluster

✓ **Steps to Deploy the Dashboard:**

1 **Deploy the Dashboard:**

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/dashboard/v2.0.0/aio/deploy/recommended.yaml
```

This creates all the dashboard resources.

2 **Create a Service Account for Access:**

```
kubectl create serviceaccount dashboard-admin-sa
kubectl create clusterrolebinding dashboard-admin-sa \
  --clusterrole=cluster-admin \
  --serviceaccount=default:dashboard-admin-sa
```

3 Get the Login Token:

```
kubectl get secret $(kubectl get serviceaccount dashboard-admin-sa -o json
path="{.secrets[0].name}") -o go-template="{{.data.token | base64decode}}"
```

Copy this token.

4 Start Proxy to Access Dashboard:

```
kubectl proxy
```

Access:

```
http://localhost:8001/api/v1/namespaces/kubernetes-dashboard/services/http
s:kubernetes-dashboard:/proxy/
```

Paste the token to log in.

✓ Deploying an Application via Dashboard:

1. Click **+ Create**.
2. You can:
 - Upload a YAML/JSON file.
 - Paste YAML.
 - Use the form to define:
 - Deployment name
 - Container image
 - Number of replicas

- Ports

3. Click **Deploy**.

4. The app starts, and you can view pods, logs, and services.

✅ **MCQ Tips Recap:**

- *kubectl proxy* starts? → The API server proxy to access dashboard.
 - *NodePort* exposes service externally.
 - *Dashboard can be used for?* → Deploying and managing resources visually.
-

Quick Recap Table

Concept	Command/Details
Create Cluster	<code>minikube start</code> (local) / <code>kubeadm init</code> (multi-node)
Create Service	<code>kubectl apply -f service.yaml</code>
Deploy Dashboard	<code>kubectl apply -f recommended.yaml</code>
Access Dashboard	<code>kubectl proxy</code> + login with token

Introduction to Software Testing

✅ **Definition:**

- **Software Testing** is the process of **evaluating a software system to identify defects and ensure that it meets requirements**.
- It is done by:
 - **Executing code**
 - Comparing actual results with expected results

✅ **Purpose:**

- Find defects before users do
- Ensure the product is reliable and works as intended

✓ Types of Testing:

- **Static Testing:** Without executing code (reviews, inspections)
- **Dynamic Testing:** By running the program (unit, integration, system testing)

✓ Analogy:

Software testing is like examining a car before delivering it to the customer—you check brakes, engine, and safety.

✓ MCQ Tip:

- *Software testing is mainly performed to?* → Detect defects.
-

Why Testing Code is Important

✓ Reasons:

1 Detect and Fix Defects Early

- Bugs cost less to fix if found early.

2 Ensure Software Works as Expected

- Meet user requirements and expectations.

3 Improve Software Quality

- Increase reliability, performance, and usability.

4 Build Customer Confidence

- Users trust software that has been tested.

5 Comply with Standards

- Many industries (medical, aviation) require thorough testing.

✓ MCQ Tip:

- *Main benefit of early testing?* → Reduces cost of defect correction.
-

Verification and Validation

Verification:

- **Are we building the product right?**
- Ensures software **conforms to specifications**.
- Activities:
 - Reviews
 - Inspections
 - Walkthroughs
- *Static process.*

Validation:

- **Are we building the right product?**
- Checks **if software meets user needs**.
- Activities:
 - Functional Testing
 - Acceptance Testing
- *Dynamic process.*

Comparison Table:

Aspect	Verification	Validation
Question Answered	Are we building it right?	Are we building the right thing?
Type	Static	Dynamic
Activities	Reviews, walkthroughs	Testing, executing code

MCQ Tip:

- *Validation ensures?* → Product meets user needs.
 - *Verification ensures?* → Product meets specs.
-

Quality Assurance vs Quality Control vs Testing

✓ Quality Assurance (QA):

- **Process-oriented**
- Focuses on **preventing defects**.
- Activities:
 - Process audits
 - Standards development
 - Training

✓ Quality Control (QC):

- **Product-oriented**
- Focuses on **identifying defects in finished products**.
- Activities:
 - Inspections
 - Reviews

✓ Testing:

- Part of **Quality Control**
- Involves executing software to find defects.

✓ Comparison Table:

Aspect	QA	QC	Testing
Focus	Process	Product	Product
Goal	Prevent defects	Identify defects	Identify defects
Activities	Process definition, audits	Reviews, inspections	Running tests

✓ MCQ Tip:

- *Testing is part of?* → Quality Control.

- *QA is mainly concerned with?* → Process improvement.
-

Principles of Software Testing

✓ Fundamental Principles:

1 Testing Shows Presence of Defects

- Testing can show defects are present, but **cannot prove no defects exist**.

2 Exhaustive Testing is Impossible

- Testing everything is not feasible.

3 Early Testing

- Testing activities should start **as early as possible**.

4 Defect Clustering

- Most defects are often found in **few modules** (Pareto principle: 80% defects in 20% components).

5 Pesticide Paradox

- Repeating same tests won't find new bugs.
- Need to **regularly review and update test cases**.

6 Testing is Context Dependent

- How you test depends on the application type.

7 Absence of Errors Fallacy

- Finding and fixing defects doesn't help if the software **doesn't meet user needs**.

✓ MCQ Tip:

- *Which principle says exhaustive testing is impossible?* → Principle 2.
 - *Pesticide paradox refers to?* → Same tests lose effectiveness over time.
-

Quick Recap Table

Concept	Key Points
Software Testing	Evaluating software to find defects
Why Testing is Important	Improves quality, detects bugs, builds confidence
Verification	Building product right (static)
Validation	Building right product (dynamic)
QA vs QC vs Testing	QA = Process, QC = Product, Testing = Execution
Testing Principles	Early testing, defect clustering, pesticide paradox

Introduction to STLC (Software Testing Life Cycle)

Definition:

- **STLC** is the **process followed to carry out testing in a systematic and planned way.**
- It defines **different phases of testing** from requirement analysis to test closure.

Phases of STLC:

Phase	What Happens?
1. Requirement Analysis	Understand and analyze testing requirements.
2. Test Planning	Create a test plan and estimate resources and timelines.
3. Test Case Design	Write detailed test cases and prepare test data.
4. Environment Setup	Prepare hardware/software environment to execute tests.
5. Test Execution	Run test cases and log defects.
6. Test Closure	Prepare test summary report, lessons learned, and finalize closure.

Analogy:

Like preparing for an exam:

- Understand syllabus → Plan study → Write practice questions → Arrange study space → Practice answers → Summarize preparation.

✓ **MCQ Tip:**

- *Test execution happens in which phase?* → Test Execution.
- *Test summary report is created in?* → Test Closure.

Introduction to V-Model (Verification and Validation Model)

✓ **Definition:**

- The **V-Model** is a **software development model** where **each development phase has a corresponding testing phase**.
- Also called the **Validation and Verification Model**.

✓ **Why "V"?**

- Because the process diagram looks like a "V".

✓ **Phases:**

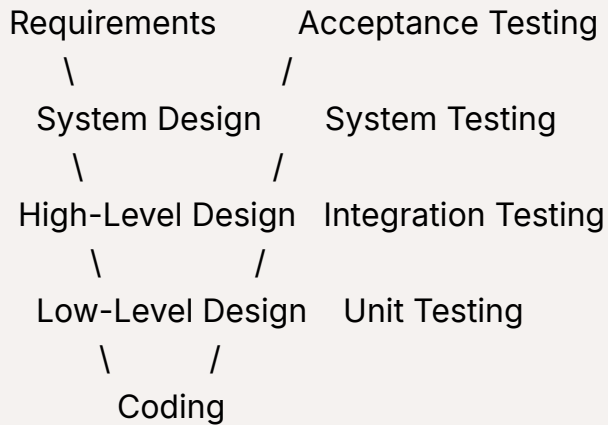
Development Phase	Corresponding Testing Phase
Requirements Analysis	Acceptance Testing
System Design	System Testing
High-Level Design	Integration Testing
Low-Level Design	Unit Testing
Coding	(Code Implementation)

✓ **Key Idea:**

- **Testing happens in parallel with development.**

- As you move down the left side, you define requirements and design.
- As you move up the right side, you validate them.

✓ V-Model Diagram (Text):



✓ Advantages:

- Simple and easy to use.
- Errors can be found early.
- Clear deliverables for each phase.

✓ MCQ Tip:

- *V-Model is also called?* → Verification and Validation Model.
- *Which testing corresponds to System Design?* → System Testing.

Types of Testing: Manual and Automation

✓ Manual Testing:

- Testing **without using any automation tools**.
- Testers execute test cases manually.
- Suitable for:

- Short-term projects
- Exploratory testing
- Usability testing

✓ **Advantages:**

- Easy to start.
- No programming skills required.

✓ **Disadvantages:**

- Time-consuming.
- Human error risk.
- Not efficient for regression testing.

✓ **Automation Testing:**

- Testing using **automation tools and scripts** to execute test cases.
- Suitable for:
 - Large projects
 - Repeated regression testing
 - Performance testing

✓ **Advantages:**

- Faster execution.
- Reusable test scripts.
- More reliable.

✓ **Disadvantages:**

- Initial setup cost high.
- Requires programming knowledge.

✓ **Comparison Table:**

Manual Testing	Automation Testing
Human-driven	Tool/script driven
Slow execution	Fast execution
No initial cost	High initial investment
Good for exploratory tests	Good for repetitive regression

✓ MCQ Tip:

- *Selenium is used for?* → Automation Testing.
- *Manual testing is preferred when?* → Requirements change frequently.

Tools Used for Automation Testing

✓ Popular Automation Tools:

Tool	Used For
Selenium	Web application testing (open source)
QTP/UFT	Functional testing by Micro Focus
JUnit/ NUnit	Unit testing for Java/.NET
TestNG	Java testing framework (advanced Selenium)
LoadRunner	Performance/load testing
Appium	Mobile application testing
JMeter	Load and performance testing (open source)
Cypress	Modern web testing (JavaScript)

✓ Example Tools and Use Cases:

1 Selenium WebDriver

- Automates browser actions.
- Supports multiple languages (Java, Python, C#).

2 JUnit

- Java unit tests.

3 Appium

- Automates iOS and Android apps.

4 LoadRunner

- Tests system performance under load.

✓ MCQ Tip:

- *Selenium is mainly used for?* → Automating web applications.
- *LoadRunner tests?* → Performance and load.

Quick Recap Table

Concept	Key Points
STLC	Steps from requirement analysis to test closure
V-Model	Parallel dev & test phases in "V" shape
Manual Testing	Human execution, exploratory, slower
Automation Testing	Tool-driven, faster, repeatable
Automation Tools	Selenium, JUnit, Appium, LoadRunner

Introduction to Testing Methods

There are **three main testing methods**:

✓ 1 White-Box Testing

Also called:

- **Clear box testing**
- **Glass box testing**
- **Structural testing**

Definition:

- Testing **internal logic and code structure**.
- Tester **knows the internal workings** of the system.

What testers do:

- Check **paths, conditions, loops** in the code.
- Design test cases to cover **all statements, branches, and conditions**.

Examples:

- Unit Testing
- Code Coverage Testing

Techniques:

- Statement Coverage
- Branch Coverage
- Path Coverage

Analogy:

| Like checking a car engine by opening the hood.

 MCQ Tip:

- *White-box testing requires?* → Knowledge of code.
-

 Black-Box Testing**Also called:**

- **Behavioral testing**
- **Opaque box testing**

Definition:

- Testing **without knowing internal code**.
- Focus is on **inputs and expected outputs**.

What testers do:

- Validate that **software meets requirements**.
- Create test cases from **specifications**.

Examples:

- Functional Testing
- System Testing
- Acceptance Testing

Techniques:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing

Analogy:

| Like checking a car by driving it, without opening the hood.

✅ MCQ Tip:

- *Black-box testing focuses on?* → Functional behavior of the application.
-

✅ 3 Grey-Box Testing

Definition:

- **Combination of white-box and black-box testing.**
- Tester has **partial knowledge of internal code**.

Why use it?

- To design better test cases with **some understanding of system internals**.

Example:

- Integration Testing with knowledge of APIs.

Analogy:

| Like knowing parts of the engine while test-driving the car.

✓ **MCQ Tip:**

- *Grey-box testing requires?* → Partial knowledge of internal design.
-

Introduction to Functional Testing

Definition:

- Testing the **functional requirements** of an application.
- Focus is on **what the system does**, not how it does it.

✓ **Key Points:**

- Based on **specifications and requirements**.
- Mostly **Black-box testing**.
- Verifies each function **produces correct output for given inputs**.

✓ **Types of Functional Testing:**

1. Unit Testing

- Testing smallest code units (functions/methods).

2. Integration Testing

- Testing combined modules together.

3. System Testing

- Testing the whole system end-to-end.

4. Acceptance Testing

- Testing by customers to approve the software.

5. Smoke Testing

- Quick checks to verify basic functionality.

6. Sanity Testing

- Focused testing of a specific area after changes.

7. Regression Testing

- Testing to ensure changes don't break existing functionality.

✓ Example:

- You enter login credentials and click "Login".
- The system checks and logs you in—**functional testing validates this behavior.**

✓ MCQ Tip:

- *Functional testing is mainly concerned with?* → Functional requirements and expected outputs.
-

Introduction to Non-Functional Testing

Definition:

- Testing the **non-functional aspects** of software—**how well it works**, rather than **what it does**.

✓ Key Points:

- Focus on **performance, usability, security, reliability**.
- Usually performed **after functional testing**.
- Evaluates **quality attributes**.

✓ Types of Non-Functional Testing:

1. Performance Testing

- Measures speed, responsiveness.

2. Load Testing

- Checks behavior under expected load.

3. Stress Testing

- Tests limits by applying extreme load.

4. Scalability Testing

- Evaluates how well the system scales up.

5. Usability Testing

- Checks user-friendliness.

6. Security Testing

- Verifies protection against threats.

7. Compatibility Testing

- Checks software on different devices/platforms.

8. Reliability Testing

- Ensures system performs consistently over time.

✅ Example:

- Testing how many users can log in at once without crashing.
- Testing how fast a page loads under heavy traffic.

✅ MCQ Tip:

- *Load and stress testing are examples of?* → Non-functional testing.

Quick Recap Table

Testing Method	Key Points
White-Box	Tests code structure; needs code access
Black-Box	Tests functionality; no code access
Grey-Box	Partial code knowledge

Testing Type	Examples
Functional Testing	Unit, Integration, System, Regression
Non-Functional Testing	Performance, Load, Security, Usability

OS 5

🕒 Created @July 10, 2025 1:27 AM

1 Introduction to Selenium

✅ What is Selenium?

- **Selenium** is a **free, open-source automation tool** used for testing web applications.
- It **mimics user actions** (click, type, select) in browsers.

✅ Components of Selenium:

- **Selenium WebDriver** – Main tool to automate browsers via code.
- **Selenium IDE** – Record & playback tool (less used for serious automation).
- **Selenium Grid** – Run tests on multiple machines in parallel.

✅ Why Selenium WebDriver?

- Supports **multiple browsers**: Chrome, Firefox, Edge.
- Supports **multiple languages**: Java, Python, C#, JavaScript.

✅ MCQ Tip:

- *Selenium WebDriver automates?* → Browser actions.

2 Using Eclipse IDE

✅ Why Eclipse?

- Eclipse is a **Java IDE** where you:
 - Write Java Selenium scripts.

- Compile and run them easily.

✅ Steps to Set Up Selenium in Eclipse:

1. Download:

- Install **Java JDK**.
- Install **Eclipse IDE**.
- Download **Selenium Java Client Library** (zip) from selenium.dev.
- Download **WebDriver binaries** (e.g., ChromeDriver).

2. Configure Project:

- Create **Java Project** in Eclipse.
- Add Selenium **JAR files to the project Build Path**.
 - Right-click project > Properties > Java Build Path > Libraries > Add External JARs.

✅ MCQ Tip:

- *Selenium needs a WebDriver binary for?* → Browser control.

Load WebDriver

✅ What is WebDriver?

- It is the **interface** that drives the browser.

✅ Sample Code:

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class MyFirstTest {
    public static void main(String[] args) {
        // Set path to ChromeDriver
        System.setProperty("webdriver.chrome.driver", "C:\\path\\to\\chromedriver.exe");
    }
}
```

```
// Create WebDriver instance
WebDriver driver = new ChromeDriver();

// Open URL
driver.get("https://www.google.com");

// Close browser
driver.quit();
}
}
```

✅ How it works:

- **System.setProperty** – Tells Java where the driver binary is.
- **new ChromeDriver()** – Launches Chrome.
- **driver.get()** – Opens web page.
- **driver.quit()** – Closes browser.

✅ MCQ Tip:

- *driver.get()* is used to? → Open URL.

Locators (Selenium Commands)

Locators help find web elements (buttons, inputs, etc.).

Selenium supports many locators:

✅ a) By ID

Finds element by its HTML ID attribute.

```
driver.findElement(By.id("username")).sendKeys("myUser");
```

✓ b) By Name

Finds element by name attribute.

```
driver.findElement(By.name("email")).sendKeys("test@example.com");
```

✓ c) By Class Name

Finds element by CSS class.

```
driver.findElement(By.className("input-class")).click();
```

✓ d) By Tag Name

Finds elements by HTML tag.

```
driver.findElement(By.tagName("button")).click();
```

✓ e) By XPath

Finds element using XPath expression.

```
driver.findElement(By.xpath("//input[@type='password']")).sendKeys("mypassword");
```

✓ Tip:

- XPath is **very powerful**, used when ID/name are missing.

✓ MCQ Tip:

- Which locator is the most robust? → XPath.
- `driver.findElement(By.id("submit"))` does? → Finds element with ID "submit".

5 Interactions with Web Elements

a) Text Box

Enter text in input fields.

```
driver.findElement(By.id("username")).sendKeys("myUser");
```

b) Radio Button

Select a radio button.

```
driver.findElement(By.id("gender-male")).click();
```

c) Check Box

Select or deselect checkbox.

```
driver.findElement(By.name("subscribe")).click();
```

d) Drop-down Selection

Use `Select` class for `<select>` tags:

```
import org.openqa.selenium.support.ui.Select;

WebElement dropdown = driver.findElement(By.id("country"));
Select select = new Select(dropdown);
select.selectByVisibleText("India");
```

e) Keyboard Actions

Using `sendKeys` for special keys:

```
driver.findElement(By.id("search")).sendKeys(Keys.ENTER);
```

f) Mouse Actions

Use `Actions` class:

```

Actions actions = new Actions(driver);
WebElement element = driver.findElement(By.id("hover"));
actions.moveToElement(element).perform(); // hover

actions.contextClick(element).perform(); // right click

```

✓ g) Multi-select

Select multiple options:

```

Select select = new Select(driver.findElement(By.id("multiSelect")));
select.selectByIndex(0);
select.selectByIndex(2);

```

6 Quick Recap Table

Interaction	Example Code
Text Box	<code>sendKeys("text")</code>
Radio Button	<code>click()</code>
Check Box	<code>click()</code>
Drop-down	<code>Select.selectByVisibleText()</code>
Keyboard Actions	<code>sendKeys(Keys.ENTER)</code>
Mouse Actions	<code>Actions.moveToElement().perform()</code>
Multi-select	<code>Select.selectByIndex()</code> multiple times

1 Introduction to Delivery Pipeline

✓ Definition:

A **Delivery Pipeline** is a set of **automated steps** that move code **from development to production**.

✓ Purpose:

- Automate **building, testing, and deploying** software.
- Ensure **faster and reliable releases**.

✓ Main Stages:

Stage	What Happens?
Build	Code compiled and packaged.
Test	Automated tests executed.
Deploy	Application deployed to target environment.

✓ Analogy:

| Like an assembly line in a factory, but for software.

✓ Key Benefits:

- Reduces manual errors.
- Accelerates delivery.
- Improves consistency.

✓ MCQ Tip:

- *Delivery pipeline automates?* → Build, Test, and Deployment.

2 Introduction to Jenkins

✓ Definition:

- **Jenkins** is an **open-source automation server** widely used for **Continuous Integration and Continuous Delivery (CI/CD)**.

✓ What Jenkins Does:

- **Pulls code** from version control (e.g., Git).
- **Builds** the application.
- **Runs tests**.
- **Deploys** the application automatically.

✅ Why Use Jenkins?

- Automate repetitive tasks.
- Integrate tools (Git, Docker, Maven).
- Monitor builds visually.

✅ Jenkins Architecture:

Component	Role
Master	Schedules jobs, manages build nodes.
Agents	Executes jobs assigned by master.

✅ How Jenkins Works:

1. Developer pushes code to Git.
2. Jenkins detects the change.
3. Jenkins triggers a **job**:
 - Pulls code.
 - Builds.
 - Runs tests.
 - Deploys.

✅ MCQ Tip:

- *Jenkins is primarily used for?* → Continuous Integration.
-

Jenkins Management

✅ Core Management Tasks:

✅ a) Installing Jenkins

- Available as:
 - **Java WAR file** (runs via `java -jar jenkins.war`).
 - **Linux packages** (e.g., `apt install jenkins`).

- **Docker container.**
 - Web UI accessible on <http://localhost:8080> by default.
-

✓ b) Configuring Jenkins

- Initial setup requires an **admin password**.
 - You install recommended **plugins** (Git, Maven, Pipeline).
 - **Global configurations:**
 - Java path.
 - Git executable path.
 - Credentials.
-

✓ c) Creating Jobs (Projects)

- **Freestyle project:**
 - Basic build steps.
- **Pipeline project:**
 - Defines pipeline as **code** (`Jenkinsfile`).

✓ Example Freestyle Job:

- Name: **BuildApp**
- Source: Git repository.
- Build step: Run Maven.
- Post-build: Archive artifacts.

✓ Example Pipeline Job:

Jenkinsfile:

```
pipeline {  
  agent any  
  stages {  
    stage('Build') {
```

```
    steps {
        echo 'Building...'
    }
}
stage('Test') {
    steps {
        echo 'Testing...'
    }
}
stage('Deploy') {
    steps {
        echo 'Deploying...'
    }
}
}
```

✓ d) Managing Plugins

- **Plugins** extend Jenkins functionality:
 - Git Plugin
 - Pipeline Plugin
 - Docker Plugin
- Manage via **Manage Jenkins > Manage Plugins**.

✓ e) Monitoring Jobs

- Jenkins provides:
 - Build history.
 - Console output logs.
 - Status badges.

✓ f) Security Management

- Configure **users and roles**:
 - **Matrix-based security** (grant permissions per user).
 - **Credentials** for accessing Git, Docker, etc.

✓ MCQ Tip:

- *Jenkins jobs can be defined as?* → Freestyle or Pipeline.

Quick Recap Table

Concept	Key Point
Delivery Pipeline	Automates build, test, deploy
Jenkins	CI/CD server automating pipelines
Jenkins Management Tasks	Install, configure, create jobs, manage plugins, secure

1 Adding a Slave Node to Jenkins

✓ Definition:

- A **slave node** (also called **agent node**) is a machine where Jenkins runs jobs.
- Helps distribute builds across multiple machines (**scaling out**).

✓ Why Use Slave Nodes?

- Run builds on different OS (Linux, Windows).
- Speed up build times by parallel execution.
- Separate resource-heavy jobs from the master.

✓ How to Add a Slave Node:

◆ Step 1: Prepare the Agent Machine

- Make sure Java is installed on the agent machine.
- Ensure network connectivity between master and agent.

◆ Step 2: Configure in Jenkins

1. Go to **Manage Jenkins > Nodes and Clouds > New Node**.
2. Enter:
 - Node name.
 - Permanent agent.
3. Configure:
 - Remote root directory (where Jenkins files will be stored on the agent).
 - Labels (to identify node purpose).
 - Number of executors (parallel jobs).
 - Launch method: typically **Launch agent via Java Web Start** or **SSH**.

◆ Step 3: Launch the Agent

- For Java Web Start:
 - Download the **agent.jar** file.
 - Run it on the agent machine:

```
java -jar agent.jar -jnlpUrl <URL> -secret <secret>
```

- For SSH:
 - Jenkins connects automatically using SSH credentials.

✓ MCQ Tip:

- *Agent nodes are used for?* → Offloading builds from master.

Building a Delivery Pipeline

✓ What Is a Delivery Pipeline?

- An **automated sequence of stages** to:
 - Build

- Test
- Deploy

✅ How to Build a Pipeline in Jenkins:

◆ Option A: Freestyle Jobs Linked via Downstream Triggers

- Create multiple jobs:
 - Build Job
 - Test Job
 - Deploy Job
- Configure **Post-build Actions**:
 - "Trigger parameterized build on other projects."

◆ Option B: Pipeline as Code (`Jenkinsfile`)

✅ Recommended

- Write your pipeline steps in Groovy DSL.
- Example `Jenkinsfile` :

```
pipeline {
  agent any

  stages {
    stage('Build') {
      steps {
        echo 'Building the application...'
        sh './build.sh'
      }
    }
    stage('Test') {
      steps {
        echo 'Running tests...'
        sh './test.sh'
      }
    }
  }
}
```

```
}
stage('Deploy') {
    steps {
        echo 'Deploying application...'
        sh './deploy.sh'
    }
}
}
```

◆ Steps to Create:

1. Create Pipeline Job:

- Jenkins dashboard > New Item > Pipeline.

2. Configure Pipeline:

- Select "Pipeline script from SCM" if using Git.
- Or paste your `Jenkinsfile` directly.

◆ Benefits:

- Full automation.
- Versioned along with code.

✅ MCQ Tip:

- *Which plugin supports pipeline as code?* → Pipeline plugin.

Selenium Integration with Jenkins

✅ Purpose:

- Automatically run Selenium tests as part of your delivery pipeline.
- Get reports in Jenkins.

✅ How It Works:

- Jenkins executes test scripts via Maven, Gradle, or direct Java commands.

- Test reports are published in Jenkins UI.

✅ Steps to Integrate Selenium:

◆ Step 1: Prepare Selenium Tests

- Write Selenium tests in Java (JUnit or TestNG).
- Store in a Git repository.
- Use Maven for dependencies.

Example Maven `pom.xml` includes:

```
<dependencies>
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.x</version>
  </dependency>
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.x</version>
  </dependency>
</dependencies>
```

◆ Step 2: Install Jenkins Plugins

- **Maven Integration plugin** (for Maven builds).
- **HTML Publisher plugin** (to display reports).

◆ Step 3: Create a Jenkins Job

1. New Item > Maven Project.
2. Configure Git repository.
3. **Goals:**

clean test

4. Post-build Actions:

- Archive JUnit/TestNG reports:

```
**/target/surefire-reports/*.xml
```

- Publish HTML reports if needed.

◆ Step 4: Schedule or Trigger

- Trigger automatically on:
 - Code push (Git hook).
 - Periodic schedule (CRON).

✓ MCQ Tip:

- *Jenkins can run Selenium tests via?* → Maven build jobs.

● Quick Recap Table

Concept	Key Points
Adding Slave Node	Config via Nodes menu, connect via JNLP/SSH
Delivery Pipeline	Automate Build → Test → Deploy stages
Selenium Integration	Use Maven jobs or pipeline to execute tests